COMP3018: Report (Task-3 Literature Review & Task-4 Programming Project)

Cognitive Robotics

Contents

1- Task (3) Literature Review

1.1. Introduction

Ageing populations and a shrinking care workforce positioned **assistive robotics** *(human-supportive robots within physical, cognitive, and social realms of impairment affecting daily-living activities)*; a prominent technological response to a widening care gap. The field spans diverse subfields; this essay however focuses on *socially* assistive robotics (*SAR*), an assistive-robot subfield in which the robot "focuses on helping human users through social rather than physical interaction" (Tapus, Matarić and Scassellati, 2007, Abstract), as here most active research and ethical tension converge.

Robots now administer medication reminders, guide rehabilitation exercises, and therapeutically companionate in clinical and domestic settings: reduced caregiver burden, improved patient outcomes; residents became "more active and more communicative, both with each other and their caregivers" (Wada and Shibata, 2007, p. 973).

However, most-current assistive robots operate at what Sciutti et al. (2023, p. 160) call the social layer: they react to immediate stimuli but lack the cognitive depth to anticipate user needs, remember past interactions, or reason about their own performance. Sciutti et al. argue that effective assistive robots should be *cognitive*: capable of "flexible, context-sensitive action, knowing what they are doing and why they are doing it." Vernon, Metta and Sandini (2007, p. 151) formalise this requirement via a "virtuous cycle that is embedded in an ongoing process of action and perception" (the agent anticipates → learns → adapts to achieve autonomy). This essay contends that assistive robotics should graduate from reactive social behaviour to cognitive capability (intelligence deployed *over* the social layer) for sustained, personalised support.

1.2. Literature Review

Cognitive robotics: defined by Sandini, Sciutti and Vernon (2021, Overview), lies at the intersection of Robotics, Artificial Intelligence, and Cognitive and Biological Sciences, integrating "sensorimotor skills, knowledge representation and reasoning, and social interaction." This interdisciplinary grounding distinguishes it from conventional robotics *(treats the robot as purely engineered)* and from social robotics *(addresses interaction behaviour without necessarily modelling cognitive processes)*. The distinction is consequential: a robot that smiles when a patient smiles is social; a robot that infers *why*, and adapts accordingly, is *cognitive*.

42 catalogued definitions of cognition (euCognition) share a common thread (Vernon, Metta and Sandini, 2007, Abstract): anticipation, learning, and adaptation, intersected with perception and action to create autonomy (Fig. 1). This cycle provides an architectural checklist for assistive robots: a system that cannot direct its gaze toward relevant stimuli and suppress irrelevant ones *(selective attention)*, anticipate the outcome of its actions *(i.e. prospection)*, learn from past interactions *(memory)*, or adapt its strategy when performance declines *(metacognition)* is, per this framework, not yet cognitive. Kotseruba and Tsotsos's (2020, Abstract) survey of 84 cognitive architectures organises them along these abilities, thus corroborating the checklist's standing in the field. Sciutti et al. (2023, p. 160) further specify that cognitive robots should "reason about their actions and modify their behavior to improve their effectiveness"; a capacity termed *theory of mind*: the agent infers another's hidden mental state.

Furthermore, memory is not treated as a single uniform store (monolithic); Laird (2012, Chapter 9) distinguishes *episodic memory (records of specific past experiences and their contextual outcomes)* from *semantic memory (general knowledge about the world, including spatial relationships and factual constraints)* in implementational terms: episodic memory is "what you 'remember' ", and semantic memory is "what you 'know' ". For example, assistive-medication robots need episodic memory to recall that a user refused medication after a restless night, and semantic memory to know certain drugs cannot also be administered.

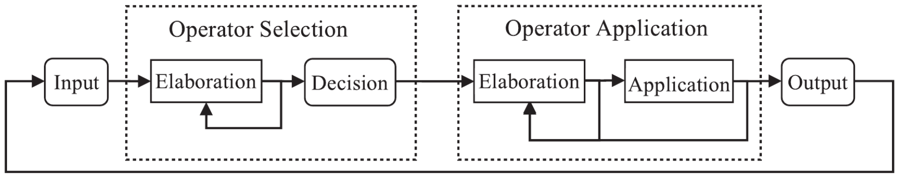

Figure 4.7
The Soar processing cycle.

Figure 1: Soar's processing cycle, reproduced from Laird (2012, fig. 4.7, p. 79), as a concrete instantiation of Vernon, Metta and Sandini's (2007) cognition cycle. Laird specifies that the cycle "consists of four phases: Input, Operator Selection, Operator Application, and Output" (Laird, 2012, p. 79), wherein Input maps to perception, Operator Selection draws on episodic and semantic memory to anticipate consequences (i.e. *prospection*), Operator Application executes the chosen behaviour through parallel rule-firing waves, and Output returns the agent to the environment whilst the loop hands control back to Input for continual adaptation. This architecture exposes the deficit of reactive systems: PARO, for instance, implements only Input $\rightarrow$ Output, lacking the substate-driven Operator Selection where deliberation over latent user states could occur. Furthermore, Laird argues that "the most significant effect of chunking is that it eliminates processing in substates for situations similar to ones experienced in the past" (Laird, 2012, p. 164); repeated deliberations compile into procedural rules; a mechanism distinct from POMDP belief-space approximation, yet complementary thereto in addressing real-time embodied operation under care-time constraints.

1.3. Applications

1.3.1 Therapeutic and Emotional Support

The PARO therapeutic seal robot is among the most-widely deployed platforms within socially assistive robotics (Tapus, Matarić and Scassellati, 2007, *.pdf*-p. 1). Wada and Shibata (2007, p. 973) demonstrate that PARO improves mood in patients with dementia, utilising tactile and auditory inputs. Clinical trials report that urinary stress indicators "significantly improved" after PARO's introduction (Wada and Shibata, 2007, p. 978, Table II), therefore the platform has been adopted in care homes.

Notwithstanding these benefits PARO operates at the reactive layer, possessing no theory of mind (cannot infer *why* a patient is agitated (loneliness, pain, confusion) nor episodic memory of *what* calmed the patient previously). A cognitively-equipped therapeutic robot, in contrast, would anticipate mood shifts (prospection), recall what calmed the patient (episodic memory), and adapt its strategy (metacognition). PARO's effectiveness plateaus because it cannot personalise; the gap is therefore clinically important. The architectural cost is: without episodic memory, the agent's perception is "limited both spatially and temporally—that is, to the here and now" (Laird, 2012, p. 233). Fong, Nourbakhsh and Dautenhahn (2003, p. 145) formalise this gap via Breazeal's taxonomy: PARO occupies the "social interface" level (human-like cues but "shallow models of social cognition"), whereas Sciutti et al.'s (2023, p. 160) vision of robots "knowing what they are doing and why" demands the *socially intelligent* level.

1.3.2 Medication Adherence and Daily Living Support

Medication non-adherence imposes substantial costs on healthcare systems, and elderly patients with polypharmacy regimens particularly are vulnerable to missed or incorrect doses. Robots in this domain face a different challenge from therapeutic companionship: trust and cognitive load are latent variables, inferred only from *noisy-behavioural signals*. Lee and See (2004, *.pdf*-p. 6) define trust as "the attitude that an agent will help achieve an individual's goals in a situation characterized by uncertainty and vulnerability"; a definition foregrounding the unobservable nature that necessitates probabilistic modelling. A user might comply with a medication prompt despite low trust (e.g. time pressure), or indeed refuse despite high trust (e.g. task complexity), and thus, the observation alone cannot reliably disambiguate the underlying state (Kaelbling, Littman and Cassandra, 1998, p. 105, *3. Partial observability*).

The Partially Observable Markov Decision Process (POMDP) provides formal machinery for this uncertainty. Chen et al. (2020, p. 6) demonstrate a Trust-POMDP wherein the robot maintains a belief distribution over trust and selects actions that maximise long-term collaboration, showing belief-space planning outperforms fixed strategies. Garcez and Lamb (2023, *.pdf*-p. 1) advocate a neuro-symbolic paradigm in which neural subsystems handle perception, and symbolic subsystems (e.g. POMDPs) handle temporal reasoning. Nikolaidis, Hsu and Srinivasa (2017, p. 625) provide empirical corroboration: in a collaborative task ($n = 69$), robots utilising mutual adaptation via a Mixed Observability MDP (modelling human adaptability as a latent variable) were rated significantly more trustworthy than fixed-policy alternatives ($U = 180$, $p = 0.048$). This aligns with Hancock et al.'s (2011, p. 522) finding that performance is the strongest trust predictor.

1.3.3 Physical Rehabilitation and Mobility

Robotic exoskeletons and assistive manipulators for stroke recovery and mobility support need to adapt not only to the patient's physical state (joint angles and muscle activation patterns) but also to their psychological state: motivation, frustration, and fatigue determine whether a patient perseveres or disengages.

Embodied cognition becomes essential. Brooks (1991, Introduction) argues that intelligence emerges from physical interaction with the environment instead of abstract representation, whereas Fong, Nourbakhsh and Dautenhahn (2003, p. 149) operationalise this as "perturbatory coupling": the more channels of mutual influence, the more embodied the system. A rehabilitation robot therefore occupies a uniquely cognitive niche, sensing the patient's body, reasoning about current capabilities, and adapting appropriately. A purely language-based or screen-based interface cannot achieve this; Matarić et al. (2007, *.pdf*-p. 7) confirm: stroke survivors engaged more with embodied robots than screen-based alternatives. Tapus, Ţăpuş and Matarić (2008, Abstract) show that embodiment alone is insufficient: adaptive personality matching (adjusting interaction distance and speed to the user's traits) further improved task performance.

1.4. Discussion

1.4.1 Challenges

Tapus, Matarić and Scassellati (2007, *.pdf*-p. 6) projected that by 2012 SAR systems would demonstrate "marked improvement in learning/training/recovery of the user"; yet PARO, the most-deployed platform nearly twenty years later, *still* cannot remember yesterday's session. Three challenges explain this. Firstly, computational intractability: solving *POMDPs* exactly is PSPACE-complete (Papadimitriou and Tsitsiklis, 1987, p. 448), and the belief simplex *(geometric space of all possible probability distributions over the hidden states)* grows exponentially with state-space dimensionality. Whilst approximate solvers such as point-based value iteration (Pineau, Gordon and Thrun, 2003, p. 1025) and online Monte-Carlo tree search (Silver and Veness, 2010, p. 1) mitigate this, real-time cognitive processing within embodied systems remains an open challenge, particularly when multiple unobserved variables (trust, load, emotion) require tracking simultaneously.

Secondly, the measurement problem: trust, cognitive load, and emotional state are not directly observable; observations thereof are noisy proxies at best. Hancock et al.'s (2011, p. 522) meta-analysis of 29 studies finds that even the strongest correlates of trust explain only moderate variance, but Broadbent, Stafford and MacDonald (2009, Abstract) note that acceptance depends on matching robot behaviour to user expectations, not trust alone. Desai et al. (2013, p. 258, Conclusion) demonstrate trust dynamics are non-linear, building slowly through consistent performance but degrading rapidly after errors; and thus a single misclassified observation can cascade. Nikolaidis, Hsu and Srinivasa (2017, p. 626), however, demonstrate that mutual adaptation partially mitigates this fragility: when the robot models human adaptability as a latent variable, trust persists even during strategy disagreements, suggesting the variance Hancock et al. report may stem from studies that treat the human as static instead of co-adaptive.

Finally, adaptation without exploitation: a robot that runs inference on cognitive load could time its medication requests to coincide with periods of high vulnerability, maximising compliance at the expense of user autonomy. The POMDP reward function should therefore encode ethical constraints alongside clinicians' objectives.

1.4.2 Ethical Implications

Assistive robots in intimate care spaces (bedrooms, bathrooms, rehabilitation clinics) continuously collect sensitive behavioural data. Facial expressions, vocal patterns, and movement trajectories constitute biometric data, yet regulatory frameworks have not kept pace with deployment. Wachter, Mittelstadt and Floridi (2017, Abstract) argue that even the General Data Protection Regulation provides no enforceable "right to explanation" of automated decisions; a gap particularly concerning in healthcare where recommendations directly affect patient wellbeing.

Moreover, over-reliance on assistive robots risks eroding functional independence. If a robot anticipates and pre-empts needs via prospection, the user may disengage from self-directed activity, contradicting the assistive mandate. Sharkey (2014, *.pdf*-p. 6) frames this via Nordenfelt's 'Dignity of Identity': "a robot that dealt impersonally with an older person, without knowing or using their name or their preferences would also be likely to negatively affect their feelings of dignity." This implies that only cognitively-equipped robots (with episodic memory of individuals) can avoid dignity violations; reactive systems such as PARO, regardless of their therapeutic benefits (Wada and Shibata, 2007, p. 973), risk infantilisation because they cannot personalise. The responsibility gap compounds this further: when a care robot administers incorrect medication, liability falls ambiguously between manufacturer, deployer, and clinician.

The ethical watchword is therefore proactive regulation: design-stage ethics anticipating failure modes, not reactive patchwork after harm. Per the embodied cognition thesis, if intelligence indeed requires a body, and that body enters the most intimate spaces of vulnerable persons, then the ethical stakes of assistive cognitive robotics are uniquely high.

1.5. Conclusion

Assistive robotics stands at an inflection point. Current systems deliver measurable benefits within narrow envelopes, yet their reactive architectures limit sustained, personalised effectiveness. The Vernon, Metta and Sandini (2007) cognition cycle provides the architectural blueprint for graduating beyond this plateau: assistive robots that anticipate (prospection), remember (episodic and semantic memory), reason about others' mental states (theory of mind), and monitor their own performance (metacognition) would mean a qualitative advance over current systems.

The neuro-symbolic paradigm offers a viable path toward this vision, as Trust-POMDP attests, with "trust dynamics" learned within symbolic planning (Chen et al., 2020, p. 6). Sciutti et al. (2023, p. 161) independently identify the integration of learning with model-based approaches as cognitive robotics' most-prominent trajectory; that this converges with Garcez and Lamb's (2023, *.pdf*-p. 1) *third wave* thesis suggests the direction is not single research group-confined. Sharkey and Sharkey (2012, *.pdf*-p. 1) caution that such systems risk replacing rather than supplementing human-care; the field should therefore pursue cognitive capability and ethical governance in concert, lest the technology displace these very carers it was meant to supplement (Fig.~2). The ultimate test, per the embodied cognition thesis, is a robot that can: sense → remember → anticipate → adapt within the physical world, whilst respecting the autonomy, and dignity, of the persons it serves.

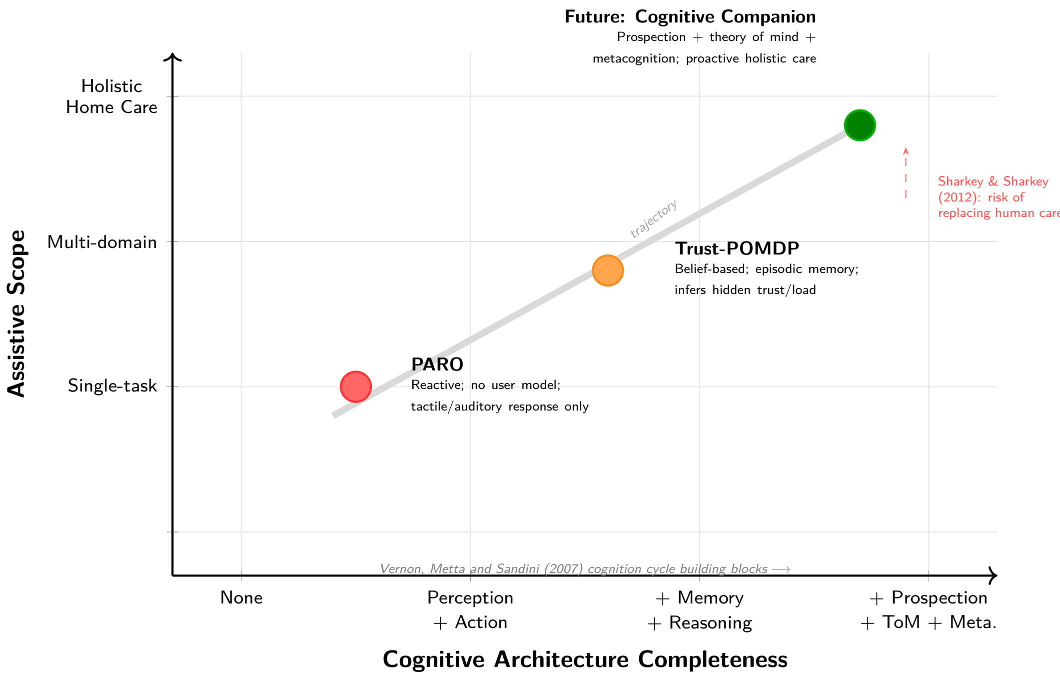

Figure 2: Trajectory of assistive robotics from reactive single-task systems (PARO) through adaptive belief-based architectures (Trust-POMDP) toward cognitively autonomous home-dwelling companions. Expanding assistive scope without expanding cognitive capability is insufficient; the diagonal trajectory requires both.

1.6. Task-3 References

1.6.1 Papers and Journal Articles

- Broadbent, E., Stafford, R. and MacDonald, B. (2009) 'Acceptance of Healthcare Robots for the Older Population: Review and Future Directions', *International Journal of Social Robotics*, 1(4), pp. 319-330. Available at: https://www.researchgate.net/publication/220397395_Acceptance_of_Healthcare_Robots_for_the_Older_Population_Review_and_Future_Directions (Accessed: 25 March 2026).
- Brooks, R. A. (1991) 'Intelligence without representation', *Artificial Intelligence*, 47(1-3), pp. 139-159. Available at: https://people.csail.mit.edu/brooks/papers/representation.pdf (Accessed: 24 March 2026).
- Chen, M., Nikolaidis, S., Soh, H., Hsu, D. and Srinivasa, S. (2020) 'Trust-Aware Decision Making for Human-Robot Collaboration: Model Learning and Planning', *ACM Transactions on Human-Robot Interaction*, 9(2), pp. 1-23. Available at: https://personalrobotics.cs.washington.edu/publications/chen2019trust.pdf (Accessed: 15 March 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) 'Impact of robot failures and feedback on real-time trust', in *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI '13)*, pp. 251-258. Available at: https://ieeexplore.ieee.org/document/6483596 (Accessed: 20 March 2026).
- Fong, T., Nourbakhsh, I. and Dautenhahn, K. (2003) 'A survey of socially interactive robots', *Robotics and Autonomous Systems*, 42(3-4), pp. 143-166. Available at: https://www.cs.cmu.edu/~illah/PAPERS/socialroboticssurvey.pdf (Accessed: 18 March 2026).
- Garcez, A. d'A. and Lamb, L. C. (2023) 'Neurosymbolic AI: The 3rd Wave', *Artificial Intelligence Review*, 56, pp. 12387-12406. Available at: https://link.springer.com/article/10.1007/s10462-023-10448-w (Accessed: 20 March 2026).
- Hancock, P. A., Billings, D. R., Schaefer, K. E., Chen, J. Y. C., de Visser, E. J. and Parasuraman, R. (2011) 'A meta-analysis of factors affecting trust in human-robot interaction', *Human Factors*, 53(5), pp. 517-527. Available at: https://journals.sagepub.com/doi/10.1177/0018720811417254 (Accessed:

15 March 2026).

- Kaelbling, L. P., Littman, M. L. and Cassandra, A. R. (1998) ‘Planning and acting in partially observable stochastic domains’, *Artificial Intelligence*, 101(1-2), pp. 99-134. Available at: https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf (Accessed: 13 March 2026).
- Kotseruba, I. and Tsotsos, J. K. (2020) ‘40 years of cognitive architectures: core cognitive abilities and practical applications’, *Artificial Intelligence Review*, 53(1), pp. 17-94. Available at: https://link.springer.com/article/10.1007/s10462-018-9646-y (Accessed: 3 May 2026).
- Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.
- Lee, J. D. and See, K. A. (2004) ‘Trust in automation: Designing for appropriate reliance’, *Human Factors*, 46(1), pp. 50-80. Available at: https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392 (Accessed: 15 March 2026).
- Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) ‘Socially assistive robotics for post-stroke rehabilitation’, *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1-9. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/ (Accessed: 25 March 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5-7), pp. 618-634. Available at: https://journals.sagepub.com/doi/10.1177/0278364917690593 (Accessed: 20 March 2026).
- Papadimitriou, C. H. and Tsitsiklis, J. N. (1987) ‘The complexity of Markov decision processes’, *Mathematics of Operations Research*, 12(3), pp. 441-450. Available at: https://web.mit.edu/jnt/www/Papers/J016-87-mdp-complexity.pdf (Accessed: 13 March 2026).
- Pineau, J., Gordon, G. and Thrun, S. (2003) ‘Point-based value iteration: An anytime algorithm for POMDPs’, in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI-03)*, pp. 1025-1030. Available at: http://www.cs.cmu.edu/~ggordon/jpineau-ggordon-thrun.ijcai03.pdf (Accessed: 24 March 2026).
- Sandini, G., Sciutti, A. and Vernon, D. (2021) ‘Cognitive Robotics’, in *Encyclopedia of Robotics*. Berlin: Springer-Verlag. Available at: http://www.vernon.eu/publications/2021_Sandini_et_al.pdf (Accessed: 3 May 2026).
- Sciutti, A., Beetz, M., Inamura, T., Korsah, A., Oh, J., Sandini, G., Shimoda, S. and Vernon, D. (2023) ‘The Present and the Future of Cognitive Robotics’, *IEEE Robotics & Automation Magazine*, 30(3), pp. 160-163. Available at: https://ieeexplore-ieee-org.plymouth.idm.oclc.org/document/10255092 (Accessed: 18 March 2026).
- Sharkey, A. (2014) ‘Robots and human dignity: A consideration of the effects of robot care on the dignity of older people’, *Ethics and Information Technology*, 16(1), pp. 63-75. Available at: https://philarchive.org/rec/SHARAH-2 (Accessed: 22 March 2026).
- Sharkey, A. and Sharkey, N. (2012) ‘Granny and the robots: ethical issues in robot care for the elderly’, *Ethics and Information Technology*, 14(1), pp. 27-40. Available at: https://philarchive.org/rec/SHAGAT (Accessed: 22 March 2026).
- Silver, D. and Veness, J. (2010) ‘Monte-Carlo planning in large POMDPs’, in *Advances in Neural Information Processing Systems (NeurIPS 23)*, pp. 2164-2172. Available at: https://proceedings.neurips.cc/paper/2010/file/edfbe1afcf9246bb0d40eb4d8027d90f-Paper.pdf (Accessed: 24 March 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35-42. Available at: https://scazlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf (Accessed: 25 March 2026).
- Tapus, A., Ţăpuş, C. and Matarić, M. J. (2008) ‘User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy’, *Intelligent Service Robotics*, 1(2), pp. 169-183. Available at: https://hal.science/hal-00770108/document (Accessed: 26 March 2026).
- Vernon, D., Metta, G. and Sandini, G. (2007) ‘A Survey of Artificial Cognitive Systems: Implications for the Autonomous Development of Mental Capabilities in Computational Agents’, *IEEE Transactions on Evolutionary Computation*, 11(2), pp. 151-180. Available at: https://www.robotcub.org/misc/papers/07_Vernon_Metta_Sandini_IEEE.pdf (Accessed: 13 March 2026).
- Wachter, S., Mittelstadt, B. and Floridi, L. (2017) ‘Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation’, *International Data Privacy Law*, 7(2), pp. 76-99. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2903469

(Accessed: 22 March 2026).

- Wada, K. and Shibata, T. (2007) 'Living with seal robots: its sociopsychological and physiological influences on the elderly at a care house', *IEEE Transactions on Robotics*, 23(5), pp. 972-980. Available at: https://ieeexplore.ieee.org/document/4339551 (Accessed: 18 March 2026).

2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)

2.1. Introduction (10%; Salman)

Gaze is an adaptive game hosting engagement coaching system that is designed to run on the pepper robot. it turns pepper into a responsive companion that plays both mathematical and letter countdown games while observing and adapting to the human. Gaze is a multi-signal emotional inference engine as the system cross validates three live data streams: facial expression recognition for emotional states, speech expression recognition and am engagement tracking feature that monitors response time, silence time and answer accuracy. Using these features helps recognising real emotional states and reduce risk or misleading readings for example a neutral face with slow responses and low accuracy means disengagement however tense expressions with fast and correct responses means focus.

Inspired by assistive robots gaze role is to keep users engaged. To ensure that it's doing its job gaze detects when attention is fading and steps in to reengage by changing difficulty level and offering hints if the user is struggling. The questions are generated dynamically using Open AI which also verifies answers and shapes the answers based on users inferred state. The speech is transcribed using whisper which allows the system to freely form answers with responses combine speech, gestures and LED feedback. All progress is saved making gaze a system that doesn't just host games but also adapts and coaches

2.2. Background (10%; Alfie)

GAZE sits within socially assistive robotics *(the deployment of robots to support users through social interaction instead of physical contact)* and affective computing, which Picard (1997, p. 1, Abstract) founds as "computing that relates to, arises from, or influences emotions"; Tapus, Matarić and Scassellati (2007, *.pdf*-p. 1) define this sub-field as systems that "assist users through social interaction." Most-current platforms react to a single input signal, suffering the single-signal problem: a facial-expression classifier misreads resting faces as displeasure; a response-time metric mistakes deliberation for disengagement. Poria et al. (2017, p. 99) report multimodal systems were "consistently (85% of systems) more accurate than their best unimodal counterparts, with an average improvement of 9.83%"; Spezialetti, Placidi and Rossi (2020, pp. 1-2, Introduction) substantiate this within HRI via reviewings of recognition systems across facial, vocal, and physiological channels. This demands multi-signal fusion (the system weighs complementary channels together instead of trusting any one alone).

GAZE's core contribution: multi-signal emotional inference, fusing six channels with derived temporal signals into a single-inferred user-state. The implementation is face-primary: voice nudges only when the face is Neutral and the MLP clears 0.9 confidence (`fearful` excluded as the silence-attractor). `gpt-5.4` generates dialogue; the symbolic AdaptiveEngine governs state inference, grounding output in Pepper's affordances (Ahn et al., 2022, *.pdf*-p. 1, Abstract); a `gpt-4.1` verifier handles answer-checking. Smedegaard (2019, p. 414, Experiential novelty) reviews the conventional assumption that engagement reflects novelty instead of sustained interest, before reframing novelty effects themselves as informative signals about user sense-making. GAZE's adaptive engine therefore aims to sustain engagement beyond the novelty phase by adaptively (dynamically) adjusting difficulty, switching games, and drafting adapted questions based on inferred user-state.

2.3. Methods & Setup (35%; Alfie)

2.3.1 System Architecture

GAZE's per-turn cycle (Fig.~3) is neuro-symbolic *(LLM does the talking, hand-coded rules do the deciding)*: `gpt-5.4` controls dialogue and decision-making whilst the AdaptiveEngine and game logic serve as callable tools, aligning with Garcez and Lamb's (2023, *.pdf*-p. 1) *third wave* paradigm wherein neural and symbolic share a structured interface (cf. Ahn et al., 2022, *.pdf*-p. 1).

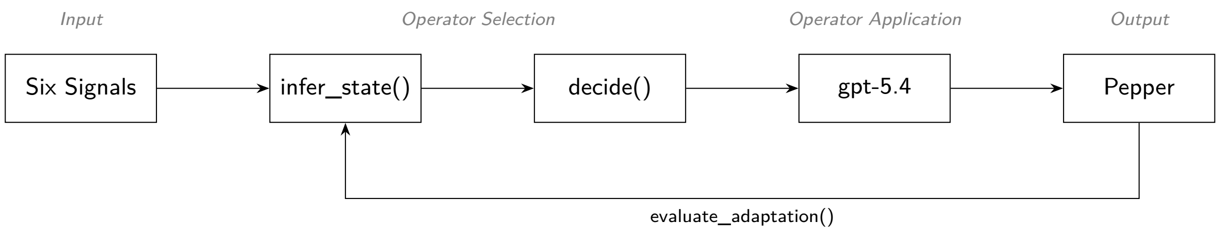

Figure 3: GAZE's per-turn cycle mapped onto Laird's (2012, fig. 4.7, p. 79) Soar phases. Each turn, six live signals fan into the AdaptiveEngine, 'gpt-5.4' generates an adapted response via function calling, and Pepper executes speech, gesture, and LED concurrently: six signals feed Input, `infer_state()` and `decide()` perform Operator Selection, *gpt-5.4* handles Operator Application; *Pepper* executes Output. Unlike Soar's implicit chunking, GAZE closes the loop via an explicit `evaluate_adaptation()` step that carries to the next turn, the loop is conversational, not question-answer.

2.3.2 Input Layer: Multimodal and Behavioural Signals

1- Facial Expression (vision-based). A pre-trained CNN (Workshop 10) classifies the user's expression into seven Ekman categories, building upon Ekman and Friesen (1971, p. 128), whose cross-cultural results show "particular facial behaviors are universally associated with particular emotions"; this taxonomy remains "still the most popular perspective for FER" (Li and Deng, 2020, p. 1). Known limitations (cultural bias, resting-face misclassification) are mitigated by combining facial with behavioural signals (§2.3.3).

The pipeline is two-stage: a Haar cascade *(OpenCV's Viola-Jones detector; locates faces via fast rectangular brightness-contrast features)* first finds the face's bounding box on a 2×-downscaled greyscale frame, then the CNN classifies emotion within the largest face ROI. Two daemon threads decouple capture from inference at $ $6.7 Hz (turn-based interaction does not warrant 30 Hz); on Pepper, frames stream over TCP via persistent `ALVideoDevice` instead of per-turn SFTP.

2- Verbal Answer (speech-based). Pepper's four-microphone polling (Listing~1) catches off-axis speakers a front-only poll would miss; `force_mono_16k_wav()` canonicalises *(forces one shape on every clip)* to mono 16 kHz. Whisper (`whisper-1`), whose models "approach [human] accuracy and robustness" (Radford et al., 2023, Abstract), serves as a delivery-aware sensor: `verbose_json` exposes `no_speech_prob`, `avg_logprob`, and `compression_ratio`, encoding *how* the user spoke alongside *what* they said. Barge-in *(letting the user interrupt Pepper mid-utterance)* was omitted: NAOqi's imperfect echo cancellation would fire false interrupts.

Listing 1: `nao_record`: calibrated-threshold silence detection polling the max energy across all four Pepper mics, with firmware fallback. Recording uses the `[1,1,1,1]` channel mask hence catching off-axis speakers a front-only poll would miss; recording terminates after 1.2 s of post-speech silence or at the adaptive hard ceiling. `force_mono_16k_wav()` downstream picks the loudest channel by RMS so Whisper, Silero-VAD, the WS-08 MLP, and `measure_volume()` all see one canonical mono 16 kHz layout.

```
def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
               record_max_secs: float = RECORD_MAX_SECS,
               silence_secs: float = SILENCE_DURATION):
    """Record audio on Pepper with dynamic silence detection.
        - poll the max energy across all four mics to stop recording early if silence detected
        - calibrated energy threshold to avoid false positives from ambient noise
```

```
        - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe fixed-duration recording to ensure the demo still works, albeit without silence detection
    """

    nao_run(ssh, f"""
from naoqi import ALProxy
import time

rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)

rec.stopMicrophonesRecording()
rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])

try:
    audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)

    speech_detected = False
    silence_start  = None
    start          = time.time()
    threshold      = {energy_threshold}

    while True:
        elapsed = time.time() - start

    # hard ceiling; never exceed max duration
        if elapsed >= {record_max_secs}:
            break

    # poll all four mics and take the MAX; a user stood to either side of Pepper registers on the side mics but not the front, so front-only polling misses them and the silence-detector stops recording mid-utterance
        try:
            energy = max(
                audio.getFrontMicEnergy(),
                audio.getLeftMicEnergy(),
                audio.getRightMicEnergy(),
                audio.getRearMicEnergy(),
            )
        except Exception:
            energy = audio.getFrontMicEnergy() # firmware fallback

    if elapsed < {RECORD_MIN_SECS}:
            # minimum recording period
            if energy > threshold:
                speech_detected = True
            time.sleep({SILENCE_POLL_SECS})
            continue

    if energy > threshold:
            speech_detected = True
            silence_start = None
        else:
            if speech_detected and silence_start is None:
                silence_start = time.time()
            if speech_detected and silence_start is not None:
                if (time.time() - silence_start) >= {silence_secs}:
                    break

```

```
    time.sleep({SILENCE_POLL_SECS})

except Exception as e:
    # firmware fallback; getFrontMicEnergy() unsupported on this Pepper
    # fall back to a safe fixed-duration recording so the demo never breaks
    print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
    time.sleep({record_max_secs})

rec.stopMicrophonesRecording()
""")
    sftp = ssh.open_sftp()
    sftp.get(REMOTE_WAV, LOCAL_WAV)
    sftp.close()
```

Transcription is gated by a five-layer defence chain (Listing~2).

Listing 2: `transcribe`: five-layer defence chain against Whisper hallucination on near-silent audio (gaze.py, lines 1532–1610). Whisper emits training-set tics on near-silent audio (CJK gibberish, prompt-primed repetition); the latter is NLG *degeneration* where models get ”stuck in repetitive loops” (Ji et al., 2023, p. 3). The chain gates: Silero VAD → Vosk wake-word → Whisper API → `no_speech_prob/avg_logprob/compression_ratio` self-signal diagnostics → normalised hallucination blacklist.

```
def transcribe(bypass_wake_word: bool = False,
               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
               record_again=None,
               max_hallucination_retries=None) -> str:
    # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
    attempts_remaining = max_hallucination_retries
    while True:
        if INPUT_IS_LOCAL and not _local_speech_detected:
            return ""

    # Silero VAD hard gate; NAO + LOCAL
        if not has_real_speech(LOCAL_WAV):
            print(" Silero VAD found no speech; skipping Whisper.")
            return ""

    # Vosk wake-word gate; bypass for name prompt
        if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
            print(" No wake-word detected; skipping Whisper.")
            return ""

    try:
            with open(LOCAL_WAV, "rb") as fh:
                resp = client.audio.transcriptions.create(
                    model="whisper-1", #
                    file=fh, #
                    response_format="verbose_json", # get self-signals for hallucination detection
                    temperature=0.0, # zero randomness
                    prompt=whisper_prompt,
                    timeout=API_TIMEOUT,
                )
            text = (getattr(resp, "text", "") or "").strip()
            print(f" Whisper raw text: {text!r}")

    # Whisper self-signals from verbose_json
            segments = getattr(resp, "segments", None) or []
            suspected_hallucination = False
```

```
        if segments:
            no_speech_vals = [s.no_speech_prob for s in segments
                              if getattr(s, "no_speech_prob", None) is not None]
            logprob_vals = [s.avg_logprob for s in segments
                            if getattr(s, "avg_logprob", None) is not None]
            compression_vals = [s.compression_ratio for s in segments
                                if getattr(s, "compression_ratio", None) is not None]
            print(f" Whisper diagnostics: no_speech={no_speech_vals}, avg_logprob={logprob_vals}, compression={compression_vals}")
            if no_speech_vals and max(no_speech_vals) > 0.6:
                print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f}); dropping {text!r}")
                return ""
            if logprob_vals and min(logprob_vals) < -1.3:
                print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f}); dropping {text!r}")
                return ""
            # repetition-loop hallucination; flag for retry path
            if compression_vals and max(compression_vals) > 2.4:
                print(f" Whisper repetition loop (max compression_ratio={max(compression_vals):.2f}); flagging {text!r} as hallucination")
                suspected_hallucination = True

# normalised hallucination blacklist + Whisper-self-signal hallucinations
        if suspected_hallucination or is_known_hallucination(text):
            print(f" Filtered Whisper hallucination: {text!r}")
            if record_again is not None and (attempts_remaining is None or attempts_remaining > 0):
                if attempts_remaining is not None: # whilst more retries remain
                    attempts_remaining -= 1
                    print(f" Disregarding hallucination; listening again (attempts left: {attempts_remaining}).")
                else:
                    print(f" Disregarding hallucination; listening again.")
                record_again()
                continue
            return ""

# Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks "Pepperoni"/"Gazebo" false positives..
        stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[,.\s]*', '', text).strip()
        return stripped
    except Exception as e:
        print(f" Whisper transcribe failed ({e}); returning empty")
        return ""
```

3- Vocal Emotion (audio-based). The same WAV passes through a pre-trained MLP (Workshop 8) *before* transcription, classifying vocal state via MFCC + chroma + mel-spectrogram features (Listing~3); fear's tremor however collapses indistinguishable from low-energy real-room speech, hence voice is engineered as a tie-breaker (§2.3.3), an architectural response to a documented limitation (El Ayadi, Kamel and Karray, 2011, p. 573, "acted emotions tend to be more exaggerated than real ones") rather than a debugging afterthought.

Listing 3: `SpeechEmotionModel.extract_features`: MFCC + chroma + mel-spectrogram feature vector matching the WS-08 training pipeline, with peak-normalisation for quieter brain-injured users (gaze.py, lines 264–292). The MLP classifies vocal state into four emotions (calm, happy, fearful, disgust); El Ayadi, Kamel and Karray (2011, p. 578) identify MFCCs as ”the most promising features” for speech-emotion recognition, supplying a second independent modality. RAVDESS is acted-speech (Williams and Stevens, cited in El Ayadi et al., 2011, p. 573, ”acted emotions tend to be more exaggerated than real ones”); WS-08 mean-pooling loses ”temporal information present in speech signals” (El Ayadi, Kamel and Karray, 2011, p. 574).

```

@staticmethod # static because it's also used independently in classify_speech_emotion()
    def extract_features(wav_path: str):
        "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
        with sf.SoundFile(wav_path) as sound_file:
            audio = sound_file.read(dtype="float32")
            sample_rate = sound_file.samplerate

    if audio.ndim > 1:
            audio = audio.mean(axis=1)

    # peak-normalise; helps quieter speakers
        audio = librosa.util.normalize(audio)

    n_fft = 2048
        if len(audio) < n_fft:
            return None

    stft = np.abs(librosa.stft(audio, n_fft=n_fft))

    mfccs = np.mean(librosa.feature.mfcc(
            y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()

    chroma = np.mean(librosa.feature.chroma_stft(
            S=stft, sr=sample_rate).T, axis=0).flatten()

    mel = np.mean(librosa.feature.melspectrogram(
            y=audio, sr=sample_rate).T, axis=0).flatten()

    return np.concatenate([mfccs, chroma, mel])
```

4- Response Time (engagement-based). A Python timer measures elapsed time from question delivery to recording completion; the Whisper call occurs *after* the timer halts, isolating deliberation time from API latency.

5- Answer Correctness (task-based). `gpt-4.1`’s `check_game_answer` evaluates the transcribed answer at a deterministic temperature; the resultant binary correctness feeds the AdaptiveEngine and the derived rolling-correctness signal.

6- Volume RMS (arousal-based). `measure_volume()` returns the audio’s RMS *(root mean square; one loudness number per chunk, computed by squaring each sample, averaging, then square-rooting)* over the mono 16 kHz WAV; LOCAL mode auto-calibrates against ambient noise whilst Pepper uses static defaults, hence volume cross-checks face evidence as a tie-breaker.

2.3.3 Process Layer: Multi-Signal State Inference

The AdaptiveEngine’s `infer_state()` classifies the user into one of five states (*Thriving*, *Comfortable*, *Struggling*, *Frustrated*, *Disengaged*) from six raw inputs supplemented by three derived temporal features *(rolling correctness over the last five rounds, consecutive-silence count, consecutive-wrong streak length)*; classification is face-primary (Fig.~4), addressing the instability Desai et al. (2013, Abstract) observe in single-signal systems. Thresholds were pilot-derived.

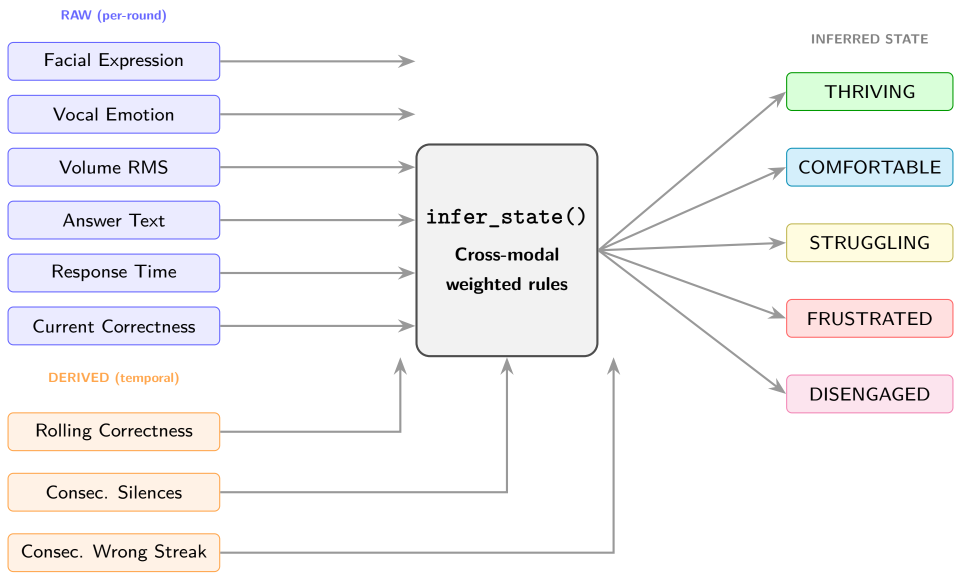

Figure 4: Multi-signal inference pipeline. Six raw inputs captured each round and three derived temporal signals computed from session history feed into `infer_state()`, which applies cross-modal rules (e.g. camera reads *Angry* but user answers fast and correctly → *Comfortable*) to classify the user into one of five states. Voice is treated as advisory instead of dominant: it nudges the state only when the face is Neutral AND the MLP's confidence clears 0.9 AND the label is not `fearful` (the silence-attractor), thus a noisy vocal label can never override stronger visual or behavioural evidence; Busso et al. (2004)'s results indeed report 85.1% unimodal accuracy for facial expression against 70.9% for speech; therein resides the multi-signal novelty.

Listing 4: Multi-signal inference rules (verbatim excerpt from `gaze.py` `infer_state()`; method-body indentation stripped). The newest implementation is face-primary: every rule fires off facial expression + behavioural signals first, voice is consulted only as a high-confidence tie-breaker at the end. Local-mode arousal calibration: `VOLUME_QUIET = max(ambient` × `2, 200)`, `VOLUME_LOUD = max(ambient` × `10, 2000)`; Pepper-mode applies static defaults (200/2000).

```

def infer_state(self, expression: str, response_time: float,
                correct: bool, answer_text: str,
                vocal_emotion: str = "neutral",
                vocal_conf: float = 0.0,
                volume_rms: float = 0.0) -> InferredState:
        """
        Infer the user's state from all signals.
        Face is primary; voice is only derived when face is Neutral and
        the voice signal is high-confidence and not "fearful" (the MLP
        collapses to "fearful" in silence).
        """
        correctness = self.rolling_correctness()
        clean = answer_text.strip().lower()
        is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no meaningful input || input matches a skip-command phrase in the set (set over list because it's faster) then indeed silent (True)

    # Arousal bounds calibrated against ambient noise
        high_arousal = volume_rms > self.VOLUME_LOUD
        low_arousal = 0 < volume_rms < self.VOLUME_QUIET

    # voice trusted only when not-fearful
```

```
        trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")

    if is_silent:
            self.consecutive_silences += 1
        else:
            self.consecutive_silences = 0
        if correct:
            self.consecutive_correct += 1
            self.consecutive_wrong = 0
        else:
            self.consecutive_wrong += 1
            self.consecutive_correct = 0

    # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted

    # thriving: good performance + fast responses
        if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
            return InferredState.THRIVING
        if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
            return InferredState.COMFORTABLE

    # disengaged: silence + slow + poor performance
        if self.consecutive_silences >= SILENCE_THRESHOLD:
            return InferredState.DISENGAGED
        if (expression == "Neutral"
                and response_time > RESPONSE_TIME_BASELINE
                and correctness < 0.5):
            return InferredState.DISENGAGED
        if (low_arousal and expression == "Neutral"
                and response_time > RESPONSE_TIME_BASELINE * 0.8):
            return InferredState.DISENGAGED

    # frustrated: negative face + poor performance
        if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
            return InferredState.FRUSTRATED
        if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
            return InferredState.FRUSTRATED
        if (high_arousal and expression in ("Angry", "Disgust", "Fear")
                and correctness < CORRECTNESS_FLOOR):
            return InferredState.FRUSTRATED

    # struggling: sadness + slow; poor correctness; fear + wrong
        if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
            return InferredState.STRUGGLING
        if correctness < CORRECTNESS_FLOOR:
            return InferredState.STRUGGLING
        if expression == "Fear" and not correct:
            return InferredState.STRUGGLING

    # 2- voice tie-breakers; only when face neutral
        if expression == "Neutral" and trust_voice:
            if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
                return InferredState.THRIVING
            if vocal_emotion == "calm" and correctness >= 0.5:
                return InferredState.COMFORTABLE

    return InferredState.COMFORTABLE # default: face gave no negative signal, performance is holding
```

The `decide()` function maps the inferred state to concrete adaptive actions, paralleling Nikolaidis, Hsu and Srinivasa's (2017, p. 625) mutual-adaptation paradigm; Table~1 summarises the mapping.

Inferred State	Difficulty	Game Switch	Hint	Tone	LED Colour
Thriving	↑ ramp up	No	No	Energetic	Green
Comfortable	↑ if >70%	No	No	Neutral	Cyan
Struggling	↓ ease off	No	Yes	Encouraging	Yellow
Frustrated	↓↓ Easy	After 3 wrong	No	Calm	Red
Disengaged	—	After 3 checkouts	No	Energetic	Magenta

Table 1: State-to-action mapping. The adaptive engine translates each inferred state into a specific combination of difficulty adjustment, game-type switch, hint provision, tone selection, and LED colour. Arrows indicate direction of difficulty change relative to the current level; the mapping applies solely to `decide()`'s engine output (the spoken-checkout path, e.g. repeated *skip*). Disengagement is flagged by three OR'd rules: 1) two consecutive silences (mic-empty or *skip/pass* checkout phrases), 2) neutral-face + response > 15 s + rolling-correctness < 50% (cognitive checkout), 3) low-volume + neutral-face + slow response (early-warning drift); the main-loop then tiers its intervention, tier-1 check-in at 3 silences and tier-2 switch at 4 each firing once per silent spell, bypassing `decide()` entirely. The graduated escalation aligns with Desai et al.'s (2013, p. 256) finding that, post-reliability-drop, "the rate of recovery is lower than it would otherwise be"; a gentle check-in therefore precedes any disruptive game-switch.

Disengagement is flagged by three OR'd rules; GAZE tiers its intervention (Table~1).

Complementing state inference, `recommend_think_budget()` sets per-turn timing thresholds *(no-speech max, silence tolerance, recording ceiling)* by combining five signals; the LLM's `waiting` flag is one among many, not the sole path. `request_more_time` extends the budget *additively*, the four behavioural signals extend independently.

2.3.4 Generate Layer: Dynamic Prompt Construction

GAZE utilises OpenAI function calling to let `gpt-5.4` decide actions via four callable tools; the LLM calls none during natural chat, sequences them during gameplay. Game-question generation runs as a separate JSON-only call with a do-not-repeat memory, preventing mode-collapsed targets *(repeated narrow outputs that ignore prompt variation)*. A deterministic `gpt-4.1` verifier (temperature 0.0) handles answer verification, reducing the hallucination risk Ji et al. (2023, p. 3) identify.

A second safeguard inside `build_signal_context()` *semantically buckets* signals (e.g. `System pacing: relaxed ↪ and patient` instead of raw `Think budget: 18s`), preventing the temperature (0.8) from echoing integers verbatim; a failure mode Ji et al. (2023, p. 3) classify under NLG hallucination. After `check_game_answer` fires, same-chain `generate_game_question` is deferred behind: `engine.decide()` updates difficulty first thus preserving adaptation across the neuro-symbolic boundary.

2.3.5 Output Layer: Aligned Multimodal Response

Speech and LED fire concurrently via threading; correct answers additionally fire a celebratory gesture. LED colours reflect the inferred state (Table~1); EarLeds turn blue during LLM inference. Before TTS, `clean_for_tts()` strips markup and applies NFKC normalisation; `split_into_sentences()` chunks at sentence boundaries with 0.4 s pauses for natural delivery. A dedicated TTS-only SSH connection isolates speech from camera + gesture traffic.

2.3.6 Adaptation Self-Evaluation and Session Persistence

`evaluate_adaptation()` compares this round's with the prior's to judge whether a previous adaptation helped; session progress saves to `gaze_save.json` every round, with a belt-and-braces auto-save every two turns in case of interruption.

2.4. Outcome & System Analysis (30%; Salman)

2.4.1 Overview

Gaze was evaluated through local mode on laptop and using the Nao robot in labs. Each session involving a participant seated opposite pepper robot engaging in a game and testing each feature and the outcome. The local mode allowed continuous iteration and was replacing the robots camera and microphone with the laptops while keeping everything else the same.

2.4.2 Local testing

The majority of the testing was done through local mode allowing to test the system calibration, recording, openAI question generation, question validation, facial expression detection and session save and resume to be tested without needing the robot. all the issues were detected and solved during the local testing like whisper hallucinations and question generation and validation as after first testing the system was showing correct to false answers so a validation function was added. After iteration testing the system repeated the same questions and to fix it `gpt-5.4` was used to ban questions that were used forcing it to create new questions and save each session instead of every 5 as some people won't play 5 questions and to make sure their progress is saved all time it was changed to save every round. One of the main set backs was hallucinations as when the mic captures ambient noise whisper would return youtube phrases from the training data rather than emptying the string this was fixed using Solero VAD as a check before whisper and banned words list for the common hallucinations however with short words like yes or fresh were sometimes dropped as logprob scores were bellow threshold to balance it the threshold was turned down from -0.85 to -1.3 after iteration testing making it rejecting hallucination from short answers.

2.4.3 Lab session testing

The testing with the robot was conducted through 6 lab sessions with the Nao robot which had hardware that the system wasn't tested on yet. The ambient noise calibration returned level of zero energy as the get front mic energy wasn't supported on the Nao robot means that silence detection never triggered and robot records the full duration around 15 seconds each rather than stopping after speech detected which caused longer responses compared to local mode. The recording max was reduced from 8 to 6 seconds to help the long replies. A hybrid mode was also introduced using external camera and microphone from the laptop for better facial expression detection and speech detection while still interacting with the robot. this solved the silence detection issue as the laptop microphone supports the energy calibration so the system can detect when the user has stopped speaking now and stop recording after it detects 2 seconds of silence rather than waiting the whole speech max duration. The hybrid mode also improved facial expression detection as the laptop camera shows a more clear image of the users face compared to the Nao robot as its important to establish the state as sit relies on facial expressions and this change resulted in better state detection giving more accurate results. During the robot testing it was noted that the robot gesture movement wasn't smooth and was too fast which was a robot safety concern as could lead to robots arm it getting stuck or hitting its body while performing a gesture. The interpolation time for all joint movements were increased from 1 second to 2-3 seconds and a safe park as a checkpoint before any movement is made its set into the safe position then goes away from body to ensure it doesn't hit then arm goes up to perform the gesture. This change made the gesture movements safer and smoother to meet safety standards. it was noted that gestures were firing too much slowing the interaction down and having more attention on gestures rather than the game. The gesture logic was reviewed then reduced so that the robot only waves at the start of the session to say hello as a welcoming and when a user gets a correct answer as a celebration he waves twice to make it different from a hello wave and at the end of a session as a goodbye. This change keeps the gesture meaningful and doesn't draw too much attention rather than the game and interrupting the conversations and responses by slowing them down with gestures.

2.5 Conclusion (10%; Alfie)

GAZE implements multi-signal emotional inference across two independent modalities *(facial expression via WS-10 CNN and vocal emotion via WS-08 MLP)* alongside speech volume/RMS, response time, answer correctness, answer text, and derived temporal signals, hence yielding a more stable user-state estimate than any single channel. The hybrid architecture (`gpt-5.4` dialogue paired with symbolic rule-based inference and deterministic `gpt-4.1` answer verification) and adaptive game-switching directly target the novelty-decay assumption Smedegaard (2019, p. 414, Experiential novelty) reviews.

Every turn, `decide()` resolves a tone label (encouraging, celebratory, calm, energetic, neutral) from the inferred state; the LLM's dialogue register thereby modulates live instead of holding a fixed persona, extending Tapus, Tapus and Mataric's (2008, Abstract) personality-matching paradigm and Kahn et al.'s (2008, pp. 97-104) design patterns for sociality from static trait-fit to dynamic state-driven adaptation.

The multi-signal approach transfers to stroke rehabilitation re-engagement (Mataric et al., 2007, *.pdf*-p. 1, Abstract *"monitoring, encouragement, and reminders"*), educational tutoring, or neurodivergent support. GAZE sits within Section 1's cognitive trajectory rather than the reactive social layer alone.

GAZE's most important limitation is epistemic: Smedegaard's (2019, p. 414, Experiential novelty) novelty-decay claim, central to the architectural rationale, is not herein empirically tested across multiple sessions; the adaptive engine *aims to* sustain engagement beyond novelty instead of demonstrating that it does. Furthermore, thresholds were derived from a single pilot, hence cross-user generalisation is unproven. The voice MLP, for its part, is the weakest channel by design, and may be quietly carrying inference noise that the face-primary rules cannot fully reject.

2.6 Task-4 References (5%)

2.6.1 Academic Books

- Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press. Available at: https://www.scribd.com/document/264059670/The-Soar-Cognitive-Architecture-The-MIT-Press-2012-John-E-Laird *FREE-TRIAL ACCESSED* (Accessed: 20 March 2026).

2.6.2 Academic Papers

- Ahn, M., Brohan, A., Brown, N., et al. (2022) 'Do As I Can, Not As I Say: Grounding Language in Robotic Affordances', in *Proceedings of the 6th Conference on Robot Learning (CoRL 2022)*, *Proceedings of Machine Learning Research*, vol. 205. Available at: https://arxiv.org/abs/2204.01691 (Accessed: 24 March 2026).
- Busso, C., Deng, Z., Yildirim, S., Bulut, M., Lee, C.M., Kazemzadeh, A., Lee, S., Neumann, U. and Narayanan, S. (2004) 'Analysis of emotion recognition using facial expressions, speech and multimodal information', in *Proceedings of the 6th International Conference on Multimodal Interfaces (ICMI '04)*. New York: ACM, pp. 205–211. Available at: https://dl.acm.org/doi/10.1145/1027933.1027968 (Accessed: 4 May 2026).
- Calvo, R.A. and D'Mello, S. (2010) 'Affect detection: An interdisciplinary review of models, methods, and their applications', *IEEE Transactions on Affective Computing*, 1(1), pp. 18–37. Available at: https://doi.org/10.1109/t-affc.2010.1 (Accessed: 14 April 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) 'Impact of robot failures and feedback on real-time trust', in *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI '13)*, pp. 251–258. Available at: https://ieeexplore.ieee.org/document/6483596 (Accessed: 20 March 2026).
- Ekman, P. and Friesen, W.V. (1971) 'Constants across cultures in the face and emotion', *Journal of Personality and Social Psychology*, 17(2), pp. 124–129. Available at: https://doi.org/10.1037/h0030377 (Accessed: 14 April 2026).
- El Ayadi, M., Kamel, M.S. and Karray, F. (2011) 'Survey on speech emotion recognition: Features, classification schemes, and databases', *Pattern Recognition*, 44(3), pp. 572–587. Available at: https://doi.org/10.1016/j.patcog.2010.09.020 (Accessed: 14 April 2026).

- Garcez, A. d'A. and Lamb, L. C. (2023) 'Neurosymbolic AI: The 3rd Wave', *Artificial Intelligence Review*, 56, pp. 12387–12406. Available at: https://link.springer.com/article/10.1007/s10462-023-10448-w (Accessed: 20 March 2026).
- Ji, Z., Lee, N., Frieske, R., et al. (2023) 'Survey of Hallucination in Natural Language Generation', *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: https://dl.acm.org/doi/10.1145/3571730 (Accessed: 22 March 2026).
- Kahn, P.H., Jr., Freier, N.G., Kanda, T., Ishiguro, H., Ruckert, J.H., Severson, R.L. and Gill, B.T. (2008) 'Design Patterns for Sociality in Human-Robot Interaction', in *Proceedings of the 3rd ACM/IEEE International Conference on Human-Robot Interaction (HRI '08)*, pp. 97–104. Available at: https://doi.org/10.1145/1349822.1349836 (Accessed: 14 April 2026).
- Li, S. and Deng, W. (2020) 'Deep Facial Expression Recognition: A Survey', *IEEE Transactions on Affective Computing*, 13(3), pp. 1195–1215. Available at: https://doi.org/10.1109/TAFFC.2020.2981446 (Accessed: 14 April 2026).
- Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) 'Socially assistive robotics for post-stroke rehabilitation', *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1–9. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/ (Accessed: 25 March 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) 'Human-robot mutual adaptation in collaborative tasks: Models and experiments', *The International Journal of Robotics Research*, 36(5–7), pp. 618–634. Available at: https://journals.sagepub.com/doi/10.1177/0278364917690593 (Accessed: 20 March 2026).
- Picard, R.W. (1997) 'Affective Computing', *MIT Media Laboratory Perceptual Computing Section Technical Report No. 321.* Available at: https://affect.media.mit.edu/pdfs/95.picard.pdf (Accessed: 14 April 2026).
- Poria, S., Cambria, E., Bajpai, R. and Hussain, A. (2017) 'A review of affective computing: From unimodal analysis to multimodal fusion', *Information Fusion*, 37, pp. 98–125. Available at: https://doi.org/10.1016/j.inffus.2017.02.003 (Accessed: 14 April 2026).
- Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) 'Robust Speech Recognition via Large-Scale Weak Supervision', in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, *Proceedings of Machine Learning Research*, 202, pp. 28492-28518. Available at: https://proceedings.mlr.press/v202/radford23a/radford23a.pdf (Accessed: 14 April 2026).
- Smedegaard, C. V. (2019) 'Reframing the Role of Novelty within Social HRI: From Noise to Information', in *Proceedings of the 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI '19)*, pp. 411–420. Available at: https://dl.acm.org/doi/10.1109/HRI.2019.8673219 (Accessed: 22 March 2026).
- Spezialetti, M., Placidi, G. and Rossi, S. (2020) 'Emotion Recognition for Human-Robot Interaction: Recent Advances and Future Perspectives', *Frontiers in Robotics and AI*, 7, Article 532279. Available at: https://www.researchgate.net/publication/347520928_Emotion_Recognition_for_Human-Robot_Interaction_Recent_Advances_and_Future_Perspectives (Accessed: 14 April 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) 'Socially assistive robotics [Grand Challenges of Robotics]', *IEEE Robotics & Automation Magazine*, 14(1), pp. 35–42. Available at: https://scazlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf (Accessed: 25 March 2026).
- Tapus, A., Ţăpuş, C. and Matarić, M. J. (2008) 'User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy', *Intelligent Service Robotics*, 1(2), pp. 169–183. Available at: https://hal.science/hal-00770108/document (Accessed: 26 March 2026).

3. Appendix

3.1 Video Demo

- YouTube link: https://youtu.be/G9-pWalwjpA

3.2 AI Declaration

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

Figure 5: Student Declaration of AI Tool use in this Assessment Table

I declare that I've used the AI tools listed below whilst preparing this assessment. I've read and understood the University of Plymouth's policy on the use of AI tools in assessment and confirm that my use falls within the coursework's allowed categories, i.e. **A2 (Planning and Structuring Projects)** and **A4 (Research Assistance)**.

AI Tool Used	Purpose of Use	Extent of Use
ChatGPT	Outlining the project structure and drafting the report hierarchy **(A2)**	Initial structuring and final outline stages for GAZE
ChatGPT	Reviewing structural alignment against grading criteria and mapping word-count budgets **(A2)**	After and midway through section-drafting
ChatGPT	Finding relevant pages to read in the paper **(A4)**	Few times if the paper is too long
ChatGPT	General conversations via web-search AI about prevalent papers to read about how the topics relates to others' studies **(A4)**	Few times at the end
OpenAI API ('gpt-5.4' & 'gpt-4.1')	Core system functionality allowed via disclosure (proposal): generate dialogue, dynamic prompts, and verifying user answers within the GAZE Task-4 Novel Contribution	API used throughout local and lab testing sessions as the system's dialogue and reasoning engine

☒ I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.

☒ I confirm that all details provided above are an accurate description of how AI was used for this assessment.

3.3 Novel Python Project: `.env` and `gaze22.1.py`

.env:

```
OPENAI_API_KEY= {get ur own key with: https://platform.openai.com/home}
NAO_IP=ROBOT_IP # hybridly doesn't need pepper but works with the blue robot we tested in the lab I believe
GAZE_LOCAL_CAMERA=true # make true if no NAO connection; false is connected to Pepper (NAO)
GAZE_LOCAL_MODE=true
```

gaze22.1.py:

```
"""
GAZE: Game-Adaptive Zone of Engagement.

A Pepper (NAO) countdown-game host. The robot adapts difficulty,
pacing and feedback per turn from six signals (each paired with the function wherein it is produced):
   1- facial expression: (CNN, WS-10)    primary     --->    capture_and_classify() / FacialExpressionModel.predict()
   2- answer correctness: (performance)  primary     --->    check_answer() (GPT-4.1 verifier)
   3- response time:   (behavioural)     primary     --->    time.time() - question_start (main loop)
   4- verbal answer:   (Whisper text)    primary     --->    transcribe() (Whisper-1 verbose_json)
   5- volume RMS:      (acoustic arousal) secondary  --->    measure_volume() (RMS over the canonical 16 kHz WAV)
   6- vocal emotion:   (MLP, WS-08)      tie-breaker; MLP collapses to "fearful" on quiet/noisy audio ---> classify_speech_emotion() --> SpeechEmotionModel.predict()
   All six fan in to: AdaptiveEngine.infer_state() -> AdaptiveEngine.decide()

NOVELTY:
Multi-signal integration: single signals are not trusted alone. Voice is only consulted when the
face is Neutral, vocal-emotion confidence is >= 0.9 and the label is not "fearful"; the MLP might
```

```
collapse to it on quiet/noisy audio.

ARCHITECTURE: gpt-5.4 converse with access to four function-calling tools:
    - generate_game_question (gpt-5.4 generates; validate_numbers_round stubbed return True)
    - check_game_answer (gpt-4.1 yes/no verifier)
    - evaluate_last_adaptation (self-evaluates previous round's strategy)
    - request_more_time (acknowledges user's request for more think-time)

Initially used a wake-word defence but became unnecessary

`AdaptiveEngine`: six signals and adapt (change based on inference) difficulty, pacing/think-budget, and game-switching:
 - gpt-4.1 for yes/no checks (check_answer, is_goodbye, resume-intent)
 - gpt-5.4 for the main converse loop and question generation.

VARIOUS MODES:
    1- LOCAL_MODE=true:
        - webcam; works with Mac and Windows (no NAO connectionc)
        - Mac/Windows mic
        - local TTS; just run entirely without robot-connection because all input/output is local to tester's computer
    2- LOCAL_MODE=false:
        - Pepper over SSH;
        - output (TTS, LEDs, gestures) on robot.

    NAO-ran code must be non-indented to avoid processing issues with Pepper (Pepper's Python 2.7 interpreter chokes on leading whitespace)

    Now, everything is ran hybridly: dashboard stats, and the camera feed is streamed over TCP to the computer for
    display and facial-expression inference. Pepper's camera is too slow for a responsive experience, and local webcam is much smoother. The microphone path is governed by LOCAL_MODE, but can be forced to the local machine with HYBRID_LOCAL_INPUT so the operator can speak from the computer whilst Pepper still does the talking.

    3- HYBRID_LOCAL_INPUT (default = True):
        - mic input is forced to the Mac despite LOCAL_MODE's preference
        - essentially the brain works locally to offload inference and display whilst Pepper handles output (TTS, LEDs, gestures)

TESTING OBSERVATIONS:
    - Pepper's onboard audio is *noisy*; Whisper (trained on podcasts, YouTube) therefore hallucinates on it and is gated by defences as follows:
      - Silero VAD
      - no_speech_prob, and a hallucination blacklist.

    - Vosk wake-word ensure (has_wake_word()) was indeed wired early-in to
    combat Whisper hallucinations on noisy Pepper audio. The
    hybrid-required switch fixed this; local Mac audio sometimes
    produces these, whereas Pepper's stream more-often does, hence
    wake-word is now not necessary (transcribe(bypass_wake_word=True)).

    - Pepper camera streaming (~10 fps) < local webcam
      (~30 fps); the code therefore runs hybridly: length-prefixed RGB
      over TCP port 9558 via pepper_video_loop/pepper_camera_receive_loop;
      detect_thread_loop runs ~6-7 Hz; display repaints ~30 fps from
      _preview_frame+_last_bbox.
    - The emotion classifiers are lightweight, not clinical instruments and thus
      robustness felt unfeasible
```

```


PER-PROPOSAL AUTHORSHIP:
    Alfie: architecture, OpenAI integration, AdaptiveEngine. facial-expression pipeline,
    Salman - game flow, gestures, LEDs, TTS pacing, save/load, testing.


LIVE PRE-DEMO CHECKLIST:
- [X] fix dashboard as it is just black when NAO
- [X] [X] eye colours should change
- [X] disregard all hallucinations until sufficent listened sentence
- [X] fix year (escape character hallucinations)
- [X] offload simpler tasks? to either computation or mini model
- [X] ensure facial expression inference ’logics commented sufficiently
- [X] ensure it notices and mitigates when user's disengaged
- [X] fix years being hallucinated
- [X] fix continouation from previous game 5 games to 3 make it says results every 3 rounds for exampe 2 out of 3 is correct and then adapts based in that

- [X] pirotise face over voice in mulit-singal inference
- [X] ensure it saves all the time not just at least 5
times

- [X] refine the arm movement its too jiterry
- [X] after escalate directly address user's disengagent
- [X] make it more random for more games as currently it keeps giving mainly the same answers even despite how hard it it is
- [X] make voice very not important
- [X] make it ask for name straight away

- [X] configure audio frequency to work better maybe for the nao (this was later resolved by hybrid-local-input switch to use local mic

- [X] make dashboard camera FPS much more fluid because its too slow now; fix was to make it stream via TCP to computer

"""

import os, re, sys, json, time, types, wave, struct, socket, textwrap, tempfile, threading, subprocess, unicodedata
import tkinter as tk
from dataclasses import dataclass, field
from enum import Enum
from typing import Optional
print("[booting...] stdlib loaded", flush=True)

import numpy as np
import cv2
from PIL import Image, ImageTk
print("[booting...] numpy + opencv loaded", flush=True)

import paramiko
print("[booting...] paramiko loaded", flush=True)

import sounddevice as sd
import librosa
import soundfile as sf
# Vosk wake-word gate
try:
```

```python
    from vosk import Model as VoskModel, KaldiRecognizer
    _vosk_import_ok = True
except ImportError:
    VoskModel = None
    KaldiRecognizer = None
    _vosk_import_ok = False
# Silero VAD; pre-Whisper speech gate
try:
    from silero_vad import load_silero_vad, read_audio, get_speech_timestamps
    _silero_import_ok = True
except ImportError:
    load_silero_vad = None
    read_audio = None
    get_speech_timestamps = None
    _silero_import_ok = False
print("[booting...] audio stack loaded", flush=True)

from dotenv import load_dotenv
from openai import OpenAI
print("[booting...] openai loaded", flush=True)

import joblib
print("[booting...] loading tensorflow (this is slow on first run)...", flush=True)
import tensorflow as tf
from tensorflow.keras.models import model_from_json
print("[booting...] tensorflow loaded", flush=True)

# ---------------------------------------- ---------------------------------------- ---------------------------------------- ----------------------------------------

# Silero VAD loader; optional dep
_silero_model = None
if _silero_import_ok:
    try:
        _silero_model = load_silero_vad()
        print("[booting] silero-vad loaded", flush=True)
    except Exception as _vad_err:
        print(f"[booting] silero-vad failed to load ({_vad_err}); VAD gate disabled", flush=True)

load_dotenv()


# NAO CONFIG
NAO_IP = os.getenv("NAO_IP", "ROBOT_IP")
NAO_USER = "nao"
NAO_PASS = "nao"
RECORD_MAX_SECS = 8 # ceiling (don't record longer than this)
RECORD_MIN_SECS = 1 # minimum recording before silence-detection
SILENCE_POLL_SECS = 0.25 # polling interval for silence detection on Pepper
SILENCE_DURATION = 1.2 # seconds of silence after speech to trigger stop
CALIBRATION_SECS = 3 # duration of start-up ambient noise calibration
ENERGY_BUFFER = 80 # margin above ambient baseline to set speech threshold
DEFAULT_ENERGY_THRESHOLD = 700 # fallback for if calibration fails
REMOTE_WAV = "/var/persistent/home/nao/input.wav"
REMOTE_IMG = "/var/persistent/home/nao/capture.jpg"
LOCAL_WAV = os.path.join(tempfile.gettempdir(), "gaze_input.wav")
LOCAL_IMG = os.path.join(tempfile.gettempdir(), "gaze_capture.jpg")
VOLUME_THRESHOLD = 700 # RMS; dashboard label only ("quiet" vs "normal"). Not a transcription gate; see NAO_MIN_RMS_TO_TRANSCRIBE.
```

```python
NAO_MIN_RMS_TO_TRANSCRIBE = 500 # near-empty WAV floor for the NAO-mode pre-transcribe gate; Silero + Whisper + blacklist do the real filtering downstream.
FACE_CONFIDENCE_THRESHOLD = 0.5 # below: face uncertain, treat neutral
VOICE_CONFIDENCE_THRESHOLD = 0.5 # threshold for vocal emotion is too uncertain; treat as neutral
SSH_TIMEOUT = 10
CMD_TIMEOUT = 60

# false when connected to pepper; true for testing when no Pepper's camera
USE_LOCAL_CAMERA = os.getenv("GAZE_LOCAL_CAMERA", "false").lower() == "true"

# uses local webcam; Mac microphone, and macOS TTS instead of Pepper-hardware
LOCAL_MODE = os.getenv("GAZE_LOCAL_MODE", "false").lower() == "true"
if LOCAL_MODE:
    USE_LOCAL_CAMERA = True

# Hybrid: TTS, LEDs and gestures stay on Pepper whereas input (camera + mic) is local
HYBRID_LOCAL_INPUT = True
INPUT_IS_LOCAL = LOCAL_MODE or HYBRID_LOCAL_INPUT

DEBUG_PREVIEW = LOCAL_MODE or HYBRID_LOCAL_INPUT
_last_rms = 0.0 # shared with recording thread for overlay
_last_emotion = "" # updated by capture_and_classify

_preview_lock = threading.Lock()
_preview_state = {"emotion": "Neutral", "confidence": 0.0}
_preview_frame = None # latest RAW BGR frame (annotation now composited in camera_refresh)
_last_bbox = None # latest detected face bbox (x, y, w, h) in raw-frame coords; None if no face
DETECT_INTERVAL_SECS = 0.15

# pre-trained model paths; checks models/ first (portable), then workshop dir (dev fallback)
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MODELS_DIR = os.path.join(SCRIPT_DIR, "models")
WORKSHOP_DIR = os.path.join(SCRIPT_DIR, "..", "..", "..", "learning", "workshops") # go backwards to root of repo then go to learning/workshops

def find_model(local_name, workshop_subpath):
    "Resolve a model local models/ dir first, then workshop fallback."
    local = os.path.join(MODELS_DIR, local_name)
    if os.path.exists(local):
        return local
    return os.path.join(WORKSHOP_DIR, workshop_subpath)

MODEL_JSON = find_model("model.json", os.path.join("[X]-facial-expression-detection", "model.json"))
MODEL_WEIGHTS = find_model("model_weights.weights.h5", os.path.join("[X]-facial-expression-detection", "model_weights.weights.h5"))
HAAR_CASCADE = find_model("haarcascade_frontalface_default.xml", os.path.join("[X]-ws-10", "haarcascade_frontalface_default.xml"))
SPEECH_MODEL = os.path.join(SCRIPT_DIR, "speech_emotion_model.pkl")
VOSK_MODEL_DIR = os.path.join(MODELS_DIR, "vosk-model-small-en-us-0.15")

# Vosk wake-word; "Pepper" or "Gaze"
_vosk_model = None
if _vosk_import_ok:
    try:
        _vosk_model = VoskModel(VOSK_MODEL_DIR)
        print("[booting] vosk wake-word model loaded", flush=True)
    except Exception as _vosk_err:
        print(f"[booting] vosk failed to load ({_vosk_err}); wake-word gate disabled", flush=True)
```

```
RESPONSE_TIME_BASELINE = 15.0 # seconds; sits below 20s recording cap
CORRECTNESS_WINDOW = 5 # rolling window size
CORRECTNESS_FLOOR = 0.4 # below: ease off
CORRECTNESS_CEILING = 0.8 # above: ramp up
SILENCE_THRESHOLD = 2 # consecutive non-responses before intervention

SAVE_FILE = os.path.join(SCRIPT_DIR, "gaze_save.json")

# ensure AI key is set
if not os.getenv("OPENAI_API_KEY", "").strip():
    raise SystemExit("ERROR: OPENAI_API_KEY not set. Add it to .env")
client = OpenAI()


class FacialExpressionModel:
    "Pre-trained CNN: 7-classed emotion classifier (48x48 greyscale input)."

    EMOTIONS = ["Angry", "Disgust", "Fear", "Happy", "Neutral", "Sad", "Surprise"]

    def __init__(self, model_json_path, model_weights_path):
        with open(model_json_path, "r") as f:
            self.model = model_from_json(f.read())
        self.model.load_weights(model_weights_path)
        self.model.make_predict_function()

    def predict(self, img):
        "Return (emotion_label, confidence) from the (1, 48, 48, 1) array."
        preds = self.model.predict(img, verbose=0)
        idx = np.argmax(preds)
        return self.EMOTIONS[idx], float(preds[0][idx])


class SpeechEmotionModel:
    "Pre-trained MLP for vocal emotion classification (WS-08)."

    EMOTIONS = ["calm", "happy", "fearful", "disgust"]

    def __init__(self, model_path):
        self.model = joblib.load(model_path)

    @staticmethod # static because it's also used independently in classify_speech_emotion()
    def extract_features(wav_path: str):
        "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
        with sf.SoundFile(wav_path) as sound_file:
            audio = sound_file.read(dtype="float32")
            sample_rate = sound_file.samplerate

        if audio.ndim > 1:
            audio = audio.mean(axis=1)

        # peak-normalise; helps quieter speakers
        audio = librosa.util.normalize(audio)

        n_fft = 2048
        if len(audio) < n_fft:
            return None

        stft = np.abs(librosa.stft(audio, n_fft=n_fft))
```

```python

        mfccs = np.mean(librosa.feature.mfcc(
            y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()

        chroma = np.mean(librosa.feature.chroma_stft(
            S=stft, sr=sample_rate).T, axis=0).flatten()

        mel = np.mean(librosa.feature.melspectrogram(
            y=audio, sr=sample_rate).T, axis=0).flatten()

        return np.concatenate([mfccs, chroma, mel])

    def predict(self, wav_path: str) -> tuple[str, float]:
        "Return (emotion_label, confidence) from a WAV file."
        features = self.extract_features(wav_path)
        if features is None:
            return "neutral", 0.0

        features = features.reshape(1, -1)
        label = self.model.predict(features)[0]
        proba = self.model.predict_proba(features)[0]
        confidence = float(np.max(proba))
        return label, confidence

def classify_speech_emotion(speech_model, wav_path: str) -> tuple[str, float]:
    "Classify the vocal emotion from a WAV file."
    if speech_model is None:
        return "neutral", 0.0
    if INPUT_IS_LOCAL and not has_real_speech(wav_path):
        return "neutral", 0.0
    try:
        return speech_model.predict(wav_path)
    except Exception as e:
        print(f" Speech emo classify failed: {e}; defaulting to neutral")
        return "neutral", 0.0


class Difficulty(Enum):
    EASY = 1
    MEDIUM = 2
    HARD = 3

class InferredState(Enum):
    THRIVING = "thriving"
    COMFORTABLE = "comfortable"
    STRUGGLING = "struggling"
    DISENGAGED = "disengaged" # thus re-engage
    FRUSTRATED = "frustrated"

class GameType(Enum):
    NUMBERS = "numbers"
    LETTERS = "letters"

BASE_SYSTEM_PROMPT = """
You are GAZE: an stroke-assitive *social-companionistic* robot running on a NAO humanoid robot.
You are a therapeutic companion first, and an engaging game host second.

CONVERSATION GUIDELINES:
- Refer to yourself ONLY in the first person ('I', 'me', 'your companion').
```

```
- Have natural, flowing conversations with the user.
- You can play countdown-style games (numbers rounds and letters rounds) when the moment feels right or the user asks, but do NOT force a game every single turn.
- Each user message includes real-time emotional signals (facial expression, vocal emotion, volume, response time). Use these signals to adapt your tone and approach naturally; doN'T mention the signals explicitly.
- Keep responses concise: 2-3 sentences maximum. Your words are spoken aloud via text-to-speech, so brevity is essential.
- If a game is active, acknowledge the user's answer before moving on.
- If the user seems disengaged, try a different topic or suggest a game. (re-engage them)
- If the user asks for more time to think, call request_more_time and respond understandably.
- If the user says the game is too hard or too easy, adjust the difficulty naturally in your next generate_game_question call.
- Be genuinely present; you are the user's social companion.
"""

 # @dataclass: typed fields auto-become __init__ params

@dataclass
class GameState:
    "Essentially tracks whether a countdown game is currently active."
    active: bool = False
    current_question: str = ""
    current_answer: str = ""
    category: str = ""
    turn_count: int = 0
    waiting: bool = False # user asked for more time to think
    last_answer_checked: bool = False # was a game answer checked this turn?
    last_answer_correct: bool = False # result of the last answer check
    # snapshot of the answered round; survives a same-chain generate_game_question overwrite
    answered_question: str = ""
    answered_answer: str = ""
    answered_game_type: Optional[GameType] = None
    answered_difficulty: Optional[Difficulty] = None

@dataclass
class RoundResult:
    "Record of a single game round."
    round_number: int
    game_type: GameType
    difficulty: Difficulty
    question: str
    user_answer: str
    correct: bool
    response_time: float
    facial_expression: str
    expression_confidence: float
    vocal_emotion: str
    vocal_emotion_confidence: float
    volume_rms: float # speech loudness (arousal signal)
    inferred_state: InferredState
    timestamp: float = field(default_factory=time.time)

@dataclass
class AdaptiveDecision:
    "Output of the adaptive engine (what to do next)"
    difficulty: Difficulty
    game_type: GameType
    inferred_state: InferredState
```

```python
    switch_game: bool
    give_hint: bool
    give_encouragement: bool
    tone: str # "encouraging" | "celebratory" | "calm" | "energetic" | "neutral"


class AdaptiveEngine:
    "Takes all six input signals and *infers* the user's real state. The adaptive engine also evaluates if its previous adaptation worked, feeding that evaluation into the next round's prompt."

    def __init__(self):
        self.history: list[RoundResult] = []
        self.current_difficulty = Difficulty.MEDIUM
        self.current_game = GameType.NUMBERS
        self.consecutive_silences = 0
        self.consecutive_correct = 0
        self.consecutive_wrong = 0
        self.games_played: dict[GameType, int] = {g: 0 for g in GameType}
        self.game_switch_count = 0
        self.adaptation_log: list[dict] = []
        self.total_correct = 0
        self.total_rounds_played = 0 # cumulative across resumed sessions
        self.best_streak = 0
        # recent-questions blocklist; cap 10
        self.recent_questions: list[str] = []
        self.recent_answers: list[str] = [] # answer-level dedup; catches mode-collapsed targets even when gpt rephrases the question text
        self.recent_game_types: list[str] = [] # game-type rotation hint; avoids 5-in-a-row Numbers games
        # adaptive think-budget; per-round baselines
        self.think_budget_secs = float(RECORD_MAX_SECS) # hard ceiling
        self.silence_tolerance_secs = float(SILENCE_DURATION) # post-speech silence
        self.no_speech_max_secs = 5.0 # give up if no speech at all


    @property
    def round_number(self) -> int:
        return len(self.history) + 1

    def rolling_correctness(self) -> float:
        recent = self.history[-CORRECTNESS_WINDOW:] # slice of 5 most recent rounds
        if not recent:
            return 0.5 # no data -> assume middle (neutral)
        return sum(1 for r in recent if r.correct) / len(recent)


    VOLUME_QUIET = 200 # below this -> low arousal (quiet/disengaged)
    VOLUME_LOUD = 2000 # above this -> high arousal (excited/frustrated)

    def infer_state(self, expression: str, response_time: float,
                    correct: bool, answer_text: str,
                    vocal_emotion: str = "neutral",
                    vocal_conf: float = 0.0,
                    volume_rms: float = 0.0) -> InferredState:
        """
        Infer the user's state from all signals.
        Face is primary; voice is only derived when face is Neutral and
        the voice signal is high-confidence and not "fearful" (the MLP
```

```python
        collapses to "fearful" in silence).
        """
        correctness = self.rolling_correctness()
        clean = answer_text.strip().lower()
        is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no meaningful input || input matches a skip-command phrase in the set (set over list because it's faster) then indeed silent (True)

        # Arousal bounds calibrated against ambient noise
        high_arousal = volume_rms > self.VOLUME_LOUD
        low_arousal = 0 < volume_rms < self.VOLUME_QUIET

        # voice trusted only when not-fearful
        trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")

        if is_silent:
            self.consecutive_silences += 1
        else:
            self.consecutive_silences = 0
        if correct:
            self.consecutive_correct += 1
            self.consecutive_wrong = 0
        else:
            self.consecutive_wrong += 1
            self.consecutive_correct = 0

        # 1- FACE-PRIMARY RULES

        # thriving: good performance + fast responses
        if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
            return InferredState.THRIVING
        if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
            return InferredState.COMFORTABLE

        # disengaged: silence + slow + poor performance
        if self.consecutive_silences >= SILENCE_THRESHOLD:
            return InferredState.DISENGAGED
        if (expression == "Neutral"
                and response_time > RESPONSE_TIME_BASELINE
                and correctness < 0.5):
            return InferredState.DISENGAGED
        if (low_arousal and expression == "Neutral"
                and response_time > RESPONSE_TIME_BASELINE * 0.8):
            return InferredState.DISENGAGED

        # frustrated: negative face + poor performance
        if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
            return InferredState.FRUSTRATED
        if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
            return InferredState.FRUSTRATED
        if (high_arousal and expression in ("Angry", "Disgust", "Fear")
                and correctness < CORRECTNESS_FLOOR):
            return InferredState.FRUSTRATED

        # struggling: sadness + slow; poor correctness; fear + wrong
        if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
            return InferredState.STRUGGLING
        if correctness < CORRECTNESS_FLOOR:
            return InferredState.STRUGGLING
```

```
        if expression == "Fear" and not correct:
            return InferredState.STRUGGLING

        # 2- voice tie-breakers; only when face neutral
        if expression == "Neutral" and trust_voice:
            if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
                return InferredState.THRIVING
            if vocal_emotion == "calm" and correctness >= 0.5:
                return InferredState.COMFORTABLE

        return InferredState.COMFORTABLE # default: face gave no negative signal, performance is holding

    # core-decision function
    def decide(self, expression: str, expression_conf: float,
               response_time: float, correct: bool,
               answer_text: str,
               vocal_emotion: str = "neutral",
               vocal_conf: float = 0.0,
               volume_rms: float = 0.0) -> AdaptiveDecision:
        "Return what to do next based on the inferred state."
        state = self.infer_state(expression, response_time, correct, answer_text,
                                 vocal_emotion, vocal_conf=vocal_conf,
                                 volume_rms=volume_rms)
        correctness = self.rolling_correctness()

        new_difficulty = self.current_difficulty
        new_game = self.current_game
        switch_game = False
        give_hint = False
        give_encouragement = False
        tone = "neutral"

        if state == InferredState.THRIVING:
            if self.current_difficulty != Difficulty.HARD:
                new_difficulty = Difficulty(self.current_difficulty.value + 1)
            tone = "energetic"
            if self.consecutive_correct >= 3:
                give_encouragement = True # acknowledge streak

        elif state == InferredState.COMFORTABLE:
            if correctness > 0.7 and self.current_difficulty != Difficulty.HARD:
                new_difficulty = Difficulty(self.current_difficulty.value + 1)
            tone = "neutral"

        elif state == InferredState.STRUGGLING:
            if self.current_difficulty != Difficulty.EASY:
                new_difficulty = Difficulty(self.current_difficulty.value - 1)
            give_hint = True
            give_encouragement = True
            tone = "encouraging"

        elif state == InferredState.FRUSTRATED:
            new_difficulty = Difficulty.EASY
            give_encouragement = True
            tone = "calm"
            if self.consecutive_wrong >= 3:
                switch_game = True
                new_game = self.pick_different_game()
```

```
    elif state == InferredState.DISENGAGED:
        tone = "energetic" # robot be engaging
        give_encouragement = True
        if self.consecutive_silences >= 3:
            switch_game = True
            new_game = self.pick_different_game()

    self.current_difficulty = new_difficulty
    if switch_game:
        self.current_game = new_game
        self.game_switch_count += 1

    self.adaptation_log.append({
        "round": self.round_number,
        "state": state.value,
        "action": {"difficulty": new_difficulty.name,
                   "switch": switch_game,
                   "hint": give_hint,
                   "encouragement": give_encouragement},
    })

    return AdaptiveDecision(
        difficulty=new_difficulty, game_type=self.current_game,
        inferred_state=state, switch_game=switch_game,
        give_hint=give_hint, give_encouragement=give_encouragement,
        tone=tone,
    )

def recommend_think_budget(self, state: InferredState, expression: str,
                           prev_response_time: float,
                           consecutive_silences: int, waiting: bool
                           ) -> tuple[float, float, float]:
    """
    Choose how long to wait for the user this turn.
    Slower or struggling users get more time, so a long pause does not
    flip the system to "disengaged" too quickly.
    Returns (no_speech_max, silence_secs, record_max_secs).
    """
    # baseline budget
    no_speech_max = 5.0
    silence_secs = float(SILENCE_DURATION)
    record_max_secs = float(RECORD_MAX_SECS)

    # round 1: stroke-recovery silence tolerance
    if not self.history:
        no_speech_max = max(no_speech_max, 7.0)
        silence_secs = max(silence_secs, 2.5)
        record_max_secs = max(record_max_secs, 15.0)

    if state in (InferredState.STRUGGLING, InferredState.FRUSTRATED, InferredState.DISENGAGED):
        no_speech_max = 8.0
        silence_secs = 2.5
        record_max_secs = 18.0

    # graduated extension before disengaged flip
    if consecutive_silences > 0:
        no_speech_max = max(no_speech_max, 5.0 + consecutive_silences * 1.5)
        silence_secs = max(silence_secs, 1.5 + consecutive_silences * 0.5)
```

```python

        if expression in ("Sad", "Fear"):
            no_speech_max = max(no_speech_max, 7.0)
            silence_secs = max(silence_secs, 2.0)

        if prev_response_time > RESPONSE_TIME_BASELINE:
            no_speech_max = max(no_speech_max, 7.0)
            silence_secs = max(silence_secs, 2.0)

        # LLM-flagged waiting: *honour* the signal but additively, so other cues stay weighted
        if waiting:
            no_speech_max += 3.0
            silence_secs += 1.0
            record_max_secs += 5.0

        # defensive ceiling 20s; under CMD_TIMEOUT
        record_max_secs = min(record_max_secs, 20.0)

        self.no_speech_max_secs = no_speech_max
        self.silence_tolerance_secs = silence_secs
        self.think_budget_secs = record_max_secs

        return no_speech_max, silence_secs, record_max_secs

    def record_round(self, result: RoundResult):
        self.history.append(result)
        self.total_rounds_played += 1
        self.games_played[result.game_type] = (
            self.games_played.get(result.game_type, 0) + 1
        )
        if result.correct:
            self.total_correct += 1
        self.best_streak = max(self.best_streak, self.consecutive_correct)

    def pick_different_game(self) -> GameType:
        if self.current_game == GameType.NUMBERS:
            return GameType.LETTERS
        return GameType.NUMBERS

    def get_session_summary(self) -> dict:
        if not self.history:
            return {"rounds": 0}
        total = len(self.history)
        correct = sum(1 for r in self.history if r.correct)
        return {
            "rounds": total,
            "correct": correct,
            "accuracy": round(correct / total, 2),
            "avg_response_time": round(sum(r.response_time for r in self.history) / total, 1),
            "games_played": {g.value: c for g, c in self.games_played.items() if c > 0},
            "game_switches": self.game_switch_count,
            "best_streak": self.best_streak,
            "final_difficulty": self.current_difficulty.name,
        }

    # Self-evaluative adaptation
    def evaluate_adaptation(self) -> Optional[str]:
        "Evaluate whether the previous round's adaptation worked."
        if len(self.adaptation_log) < 2 or len(self.history) < 2:
```

```
            return None

        prev_strategy = self.adaptation_log[-2]
        curr_strategy = self.adaptation_log[-1]
        prev_round = self.history[-2]
        curr_round = self.history[-1]

        prev_state = prev_strategy["state"]
        curr_state = curr_strategy["state"]
        prev_action = prev_strategy["action"]

        evaluations = []

        if prev_state in ("struggling", "frustrated"):
            if curr_round.correct and not prev_round.correct:
                evaluations.append("Previous adaptation WORKED: lowered difficulty and user answered correctly this round (was incorrect before).")
            elif not curr_round.correct:
                evaluations.append("Previous adaptation DID NOT HELP YET: user still struggling despite easier difficulty. Consider providing more support.")

        # Did a difficulty increase overshoot for a thriving user?
        if prev_state == "thriving" and prev_action["difficulty"] == "HARD":
            if curr_round.correct:
                evaluations.append("Previous adaptation WORKED: increased difficulty and user is still performing well.")
            elif not curr_round.correct:
                evaluations.append("Previous adaptation OVERSHOT: increased difficulty but user got it wrong. Might need to ease back.")

        # Did a re-engagement attempt work for a disengaged user?
        if prev_state == "disengaged":
            if curr_state != "disengaged":
                evaluations.append(f"Previous adaptation WORKED: user was disengaged but is now {curr_state}. Re-engagement WAS effective.")
            else:
                evaluations.append("Previous adaptation DID NOT HELP: user remains disengaged. Try a different approach or switch game type.")

        if prev_action.get("switch"):
            if curr_state in ("thriving", "comfortable"):
                evaluations.append(
                    "Game switch WORKED: user transitioned to a positive state."
                )
            elif curr_state in ("struggling", "frustrated", "disengaged"):
                evaluations.append(
                    "Game switch DID NOT HELP: user is still in a negative state."
                )

        if (prev_action.get("encouragement")
                and prev_state in ("struggling", "frustrated")
                and curr_round.response_time < prev_round.response_time):
            evaluations.append(
                "Encouragement appears effective: user responded faster this round."
            )

        if not evaluations:
            return None

```

```python
        return ("Adaptation evaluation from previous round:\n"
                + "\n".join(f"- {e}" for e in evaluations))


GAME_DESCRIPTIONS = {
    GameType.NUMBERS: """
    a Countdown-style numbers round. Pick SIX numbers by sampling from {1 to 100}; vary the mix each round so no two rounds in a row feel identical.
    Pick a TARGET in the range 50-999 (anywhere in that range, NOT always around 100), reachable from the chosen six via +, -, *, /.
    The user must combine the given numbers with those operators to reach the target. Each number may be used at most once and it should be humanly solveable""",
    GameType.LETTERS: """
    a Countdown-style letters round: give the user a set of 9 random letters (a mix of vowels and consonants; vary the letter set every round) and ask them to form the longest word possible using only those letters.
    Each letter can only be used once""",
}

DIFFICULTY_DESCRIPTIONS = {
    Difficulty.EASY: "easy (straightforward, common knowledge, single-step)",
    Difficulty.MEDIUM: "medium (requires some thought, moderately specific)",
    Difficulty.HARD: "hard (obscure, multi-step, requires deep knowledge)",
}

# SSH AND PEPPER ROBOT HELPERS: NON-INDENTED PYTHON STRINGS
# (adapted from lab-robot-code-fin.py i.e. when
# we tested OpenAI on the NAO robot perhaps too early)

def ssh_connect():
    "Open SSH connection to Pepper and return the client."
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(NAO_IP, username=NAO_USER, password=NAO_PASS, timeout=SSH_TIMEOUT)
    return ssh

def nao_run(ssh, code):
    "Execute a Python 2 snippet on Pepper via SSH."
    escaped = code.replace("'", "'\\''")
    try:
        _, stdout, _ = ssh.exec_command(f"python -c '{escaped}'", timeout=CMD_TIMEOUT)
        return stdout.read().decode().strip()
    except Exception as e:
        print(f" Pepper SSH exec_command dropped: {e}")
        return ""

#listens silent
#not triggered by noise
def nao_calibrate_ambient(ssh) -> int:
    "Calibrate the mic energy threshold to the room."
    try:
        # col 0 only; Pepper rejects indented python -c
        raw = nao_run(ssh, f"""
from naoqi import ALProxy
import time

audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
samples = []
start = time.time()
```

```
while (time.time() - start) < {CALIBRATION_SECS}:
    # all four mics; side speakers else missed
    try:
        front = audio.getFrontMicEnergy()
        left = audio.getLeftMicEnergy()
        right = audio.getRightMicEnergy()
        rear = audio.getRearMicEnergy()
        samples.append(max(front, left, right, rear))
    except Exception:
        # older firmwares may expose only getFrontMicEnergy; fall back gracefully
        samples.append(audio.getFrontMicEnergy())
    time.sleep(0.2)

if samples:
    avg = sum(samples) / len(samples)
    print(int(avg))
else:
    print(0)
""")
        ambient = int(raw) if raw.strip().isdigit() else 0
        threshold = ambient + ENERGY_BUFFER
        print(f" Ambient energy: {ambient}, speech threshold: {threshold}")
        return threshold
    except Exception as e:
        print(f" Calibration failed ({e}), using default threshold: {DEFAULT_ENERGY_THRESHOLD}")
        return DEFAULT_ENERGY_THRESHOLD

def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
               record_max_secs: float = RECORD_MAX_SECS,
               silence_secs: float = SILENCE_DURATION):
    """Record audio on Pepper with dynamic silence detection.
        - get robot's front microphone (getFrontMicEnergy) to stop recording early if silence detected
        - calibrated energy threshold to avoid false positives from ambient noise
        - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe fixed-duration recording to ensure the demo still works, albeit without silence detection
    """
    # fixed-duration fallback for old firmware
    nao_run(ssh, f"""
from naoqi import ALProxy
import time

rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)

rec.stopMicrophonesRecording()
rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])

try:
    audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)

    speech_detected = False
    silence_start = None
    start = time.time()
    threshold = {energy_threshold}

    while True:
        elapsed = time.time() - start

```

```
        # hard ceiling; never exceed max duration
        if elapsed >= {record_max_secs}:
            break

        # all four mics; side speakers else missed
        try:
            energy = max(
                audio.getFrontMicEnergy(),
                audio.getLeftMicEnergy(),
                audio.getRightMicEnergy(),
                audio.getRearMicEnergy(),
            )
        except Exception:
            energy = audio.getFrontMicEnergy() # firmware fallback

        if elapsed < {RECORD_MIN_SECS}:
            # minimum recording period
            if energy > threshold:
                speech_detected = True
            time.sleep({SILENCE_POLL_SECS})
            continue

        if energy > threshold:
            speech_detected = True
            silence_start = None
        else:
            if speech_detected and silence_start is None:
                silence_start = time.time()
            if speech_detected and silence_start is not None:
                if (time.time() - silence_start) >= {silence_secs}:
                    break

        time.sleep({SILENCE_POLL_SECS})

except Exception as e:
    # firmware fallback; if getFrontMicEnergy() unsupported, fixed-duration recording keeps the demo alive
    print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
    time.sleep({record_max_secs})

rec.stopMicrophonesRecording()
""")
    sftp = ssh.open_sftp()
    sftp.get(REMOTE_WAV, LOCAL_WAV)
    sftp.close()
    # no gain stage; amplified room noise

    # WAV layout diagnostic; four-mic recording may land multi-channel
    try:
        with wave.open(LOCAL_WAV, "rb") as wf:
            secs = wf.getnframes() / float(wf.getframerate())
            print(f" WAV debug: channels={wf.getnchannels()}, rate={wf.getframerate()}, frames={wf.getnframes()}, secs={secs:.2f}")
    except Exception as e:
        print(f" WAV debug failed: {e}")

    # collapse to mono 16 kHz so downstream gates see a canonical layout
    force_mono_16k_wav(LOCAL_WAV)

```

```
def force_mono_16k_wav(path: str) -> None:
    "Convert `path` to mono 16 kHz int16 PCM."
    try:
        audio, sr = sf.read(path, dtype="float32")
        if audio.size == 0:
            return
        if audio.ndim > 1:
            # loudest mic; mics not phase-aligned
            rms_by_channel = np.sqrt(np.mean(audio.astype(np.float64) ** 2, axis=0))
            audio = audio[:, int(np.argmax(rms_by_channel))]
        if sr != LOCAL_SAMPLE_RATE:
            audio = librosa.resample(audio, orig_sr=sr, target_sr=LOCAL_SAMPLE_RATE)
        audio = np.clip(audio, -1.0, 1.0)
        sf.write(path, audio, LOCAL_SAMPLE_RATE, subtype="PCM_16")
    except Exception as e:
        print(f" Mono conversion skipped ({e})")

 #responses arrive in one block no splitting
#delivered with no pauses
def split_into_sentences(text: str) -> list[str]:
    "Split dialogue into sentences for speech delivery."
    raw_segments = re.split(r'(?<=[.!?])\s+', text.strip())
    sentences = []
    for seg in raw_segments:
        for line in seg.split("\n"):
            cleaned = line.strip()
            if cleaned:
                sentences.append(cleaned)
    return sentences if sentences else [text.strip()]

_TTS_REPLACEMENTS = { # prevent unicode characters from breaking Pepper Text-To-Speech
    "‘": "'", "’": "'",
    """: '"', """: '"',
    "-": "-", "-": "-", "-": "-",
    "…": "...", " ": " ",
}

def clean_for_tts(text: str) -> str:
    "Strip gesture tags and Unicode escapes for Pepper TTS."
    text = text or ""
    text = re.sub(r'\[gesture:\w+\]', '', text)
    text = unicodedata.normalize("NFKC", text)
    for bad, good in _TTS_REPLACEMENTS.items():
        text = text.replace(bad, good)
    # animated-speech tags
    text = re.sub(r'\^[A-Za-z_]+(?:\([^)]*\))?', '', text)
    # ALTextToSpeech tags
    text = re.sub(r'\\[A-Za-z]{2,8}=[^\\]*\\', '', text)
    # literal \uXXXX leftovers
    text = re.sub(r'\\u[0-9a-fA-F]{4}', '', text)
    # control characters
    text = re.sub(r'[\x00-\x1f\x7f-\x9f]', ' ', text)
    return re.sub(r'\s+', ' ', text).strip()

#single ssh calls
def nao_say(ssh, text):
    "Speak text on Pepper with sentence-level pausing."
    text = clean_for_tts(text)
    sentences = split_into_sentences(text)
```

```
    safe_sentences = json.dumps(sentences)

    nao_run(ssh, f"""
from naoqi import ALProxy
import time

try:
    tts = ALProxy("ALTextToSpeech", "127.0.0.1", 9559)
    sentences = {safe_sentences}

    for i, sentence in enumerate(sentences):
        tts.say(sentence)
        if i < len(sentences) - 1:
            time.sleep(0.4)
except Exception as e:
    print(" [TTS failed] " + str(e))
""")

#animations with speech
def nao_say_animated(ssh, text):
    "Try animated speech; fall back to plain TTS."
    safe = json.dumps(text)
    try:
        nao_run(ssh, f"""
from naoqi import ALProxy
ALProxy("ALAnimatedSpeech","127.0.0.1",9559).say({safe})
""")
    except Exception as e:
        print(f" Animated speech failed mid-sentence: {e}")
        nao_say(ssh, text)

def nao_capture_image(ssh):
    "Capture a photo from Pepper's camera and download it. Kept as a fallback for one-off stills; the live dashboard now uses pepper_video_loop()."
    nao_run(ssh, f"""
from naoqi import ALProxy
pc = ALProxy("ALPhotoCapture","127.0.0.1",9559)
pc.setResolution(2)
pc.setPictureFormat("jpg")
pc.takePicture("{os.path.dirname(REMOTE_IMG)}/", "{os.path.splitext(os.path.basename(REMOTE_IMG))[0]}")
""")
    sftp = ssh.open_sftp()
    sftp.get(REMOTE_IMG, LOCAL_IMG)
    sftp.close()

PEPPER_VIDEO_PORT = 9558
_pepper_video_channel = None

def pepper_video_loop(ssh):
    """New persistent ALVideoDevice on Pepper streaming length-prefixed RGB frames over
      TCP (Transmission Control Protocol)"""
    global _pepper_video_channel

    code = textwrap.dedent('''
        from naoqi import ALProxy
        import socket, struct, time

        HOST = "0.0.0.0"
```

```
        PORT = {port}

        cam = ALProxy("ALVideoDevice", "127.0.0.1", 9559)
        # camera_id=0 (top), resolution=1 (320x240), colour_space=11 (RGB), fps=10
        name = cam.subscribeCamera("gaze_video", 0, 1, 11, 10)

        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((HOST, PORT))
        server.listen(1)

        conn, addr = server.accept()

        try:
            while True:
                img = cam.getImageRemote(name)
                if img is None:
                    time.sleep(0.05)
                    continue
                width = img[0]
                height = img[1]
                data = img[6]
                payload = struct.pack("!II", width, height) + data
                conn.sendall(struct.pack("!I", len(payload)) + payload)
                time.sleep(0.1)
        finally:
            try:
                cam.unsubscribe(name)
            except Exception:
                pass
            try:
                conn.close()
            except Exception:
                pass
            server.close()
    ''').format(port=PEPPER_VIDEO_PORT)

    escaped = code.replace("'", "'\\''")
    # async; nao_run() reads stdout, would otherwise block
    _stdin, stdout, _stderr = ssh.exec_command(f"python -c '{escaped}'")
    _pepper_video_channel = stdout.channel # keep referenced so Paramiko doesn't GC the channel out from under the remote process


def _recv_exact(sock, n):
    "Receive exactly n bytes; raise if the socket closes mid-frame."
    data = b""
    while len(data) < n:
        packet = sock.recv(n - len(data))
        if not packet:
            raise ConnectionError("Socket closed whilst receiving Pepper camera frame")
        data += packet
    return data


def pepper_camera_receive_loop(pepper_ip: str):
    "Pull length-prefixed RGB frames off Pepper:9558 into _preview_frame for detect_thread_loop to consume; ten 1-second connect-retries because Pepper-side bind() needs a beat to land after exec_command."
```

```
    global _preview_frame

    sock = None
    for attempt in range(10):
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.connect((pepper_ip, PEPPER_VIDEO_PORT))
            print(f" Pepper video socket connected on attempt {attempt + 1}.")
            break
        except (ConnectionRefusedError, OSError) as e:
            print(f" Pepper video connect attempt {attempt + 1} failed ({e}); retrying...")
            try:
                sock.close()
            except Exception:
                pass
            sock = None
            time.sleep(1.0)
    if sock is None:
        print(" Pepper video stream unreachable; dashboard will stay black.")
        return

    try:
        while True: # whilst running
            size_raw = _recv_exact(sock, 4)
            size = struct.unpack("!I", size_raw)[0]
            payload = _recv_exact(sock, size)
            width, height = struct.unpack("!II", payload[:8])
            rgb_bytes = payload[8:]
            rgb = np.frombuffer(rgb_bytes, dtype=np.uint8).reshape((height, width, 3))
            frame = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR) # rest of pipeline is BGR (OpenCV convention)
            with _preview_lock:
                _preview_frame = frame
    except Exception as e:
        print(f" Pepper video receiver loop exited: {e}")
    finally:
        try:
            sock.close()
        except Exception:
            pass

def nao_track_face(ssh, enable=True):
    "Toggle face tracking on Pepper."
    try:
        if enable:
            nao_run(ssh, """
from naoqi import ALProxy
ALProxy("ALFaceDetection","127.0.0.1",9559).subscribe("gaze_face")
t = ALProxy("ALTracker","127.0.0.1",9559)
t.registerTarget("Face", 0.1)
t.track("Face")
""")
        else:
            nao_run(ssh, """
from naoqi import ALProxy
t = ALProxy("ALTracker","127.0.0.1",9559)
t.stopTracker()
t.unregisterAllTargets()
try:
```

```python
    ALProxy("ALFaceDetection","127.0.0.1",9559).unsubscribe("gaze_face")
except Exception as e:
    print(" [Face unsubscribe failed] " + str(e))
""")
    except Exception as e:
        print(f" Face tracking unavailable: {e}")

#leds
def nao_set_leds(ssh, group, colour, duration=1.0):
    try:
        nao_run(ssh, f"""
from naoqi import ALProxy
ALProxy("ALLeds","127.0.0.1",9559).fadeRGB("{group}", {colour}, {duration})
""")
    except Exception as e:
        print(f" LED set ignored: {e}")

# LOCAL MODE HELPERS

LOCAL_SAMPLE_RATE = 16000 # Whisper expects 16 kHz; we resample from native rate

LOCAL_SILENCE_RMS = 40 # default RMS; overridden by local_calibrate_ambient()
_local_speech_detected = False # set by local_record(); used as transcription gate
LOCAL_SILENCE_SECS = 1.2 # seconds of post-speech silence to stop recording; 1.5 →1.2 (aphasia-safe)
LOCAL_MIN_SECS = 1.0 # minimum recording before silence detection kicks in
LOCAL_NO_SPEECH_MAX = 3.0 # stop if no speech detected at all after this many seconds; 5.0 →3.0 (dominant lag)
LOCAL_ENERGY_BUFFER = 60 # margin above ambient baseline for speech detection; 50 →25 to catch quiet speech

def local_calibrate_ambient() -> int:
    "Calibrate the local-testing (Mac) mic's ambient noise level; mirrors nao_calibrate_ambient() so LOCAL_MODE testing behaves like the robot."
    global LOCAL_SILENCE_RMS
    dev_info = sd.query_devices(kind="input")
    dev_index = dev_info["index"]
    native_rate = int(dev_info["default_samplerate"])
    chunk_size = int(native_rate * 0.2)
    samples = []

    print(f"Calibrating Mac mic (stay quiet for {CALIBRATION_SECS}s)...")

    def cb(indata, frames, time_info, status): # get stable baseline from average RMS
        rms = (np.mean(indata.astype(np.float64) ** 2)) ** 0.5
        samples.append(rms)

    with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
                        blocksize=chunk_size, device=dev_index, callback=cb):
        time.sleep(CALIBRATION_SECS)

    if samples:
        ambient = int(sum(samples) / len(samples))
    else:
        ambient = 0

    threshold = ambient + LOCAL_ENERGY_BUFFER
    LOCAL_SILENCE_RMS = threshold

```

```
    # 500 was too high for MacBook mic
    global VOLUME_THRESHOLD
    VOLUME_THRESHOLD = max(LOCAL_SILENCE_RMS * 4, 100)

    # scale to room; static thresholds confuse loud/quiet rooms
    AdaptiveEngine.VOLUME_QUIET = max(ambient * 2, 200) # 200 = absolute quiet floor
    AdaptiveEngine.VOLUME_LOUD = max(ambient * 10, 2000)

    print(f" Ambient RMS: {ambient}, silence threshold: {threshold}, transcription gate: {VOLUME_THRESHOLD}")
    print(f" Arousal thresholds: QUIET={AdaptiveEngine.VOLUME_QUIET}, LOUD={AdaptiveEngine.VOLUME_LOUD}")
    return threshold

def local_record(max_secs: float = RECORD_MAX_SECS,
                 no_speech_max: float = LOCAL_NO_SPEECH_MAX,
                 silence_secs: float = LOCAL_SILENCE_SECS):
    "Record audio from the Mac's built-in microphone to LOCAL_WAV with silence detection mirroring Pepper's dynamic recording."
    # select the default input device
    dev_info = sd.query_devices(kind="input")
    dev_index = dev_info["index"]
    native_rate = int(dev_info["default_samplerate"])
    sd.default.device = (dev_index, None)
    chunk_size = int(native_rate * SILENCE_POLL_SECS)

    buffer = []
    speech_detected = False
    silence_start = None
    elapsed = 0.0

    print(f" Recording from Mac mic (up to {max_secs}s, stops after silence)...")

    def callback(indata, frames, time_info, status):
        buffer.append(indata.copy())

    for attempt in range(2): #one retry if PortAudio error (if the device briefly is unavailable after Pepper usage)
        try:
            with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
                                blocksize=chunk_size, callback=callback):
                while elapsed < max_secs:
                    time.sleep(SILENCE_POLL_SECS)
                    elapsed += SILENCE_POLL_SECS

                    if not buffer:
                        continue
                    data = buffer[-1]

                    rms = (np.mean(data.astype(np.float64) ** 2)) ** 0.5 # compute RMS of the latest chunk for real-time feedback
                    global _last_rms
                    _last_rms = rms                          # '' (U+2593 DARK SHADE) and '' (U+2591 LIGHT SHADE) extracted from Unicode 1.1 Block Elements
                    print(f"\r  [{elapsed:.1f}s] RMS: {rms:.0f} {'' if rms > LOCAL_SILENCE_RMS else ''}", end="", flush=True)

                    if elapsed < LOCAL_MIN_SECS:
                        if rms > LOCAL_SILENCE_RMS:
```

```
                        speech_detected = True
                    continue

                if not speech_detected and elapsed >= no_speech_max:
                    break

                if rms > LOCAL_SILENCE_RMS:
                    speech_detected = True
                    silence_start = None
                else:
                    if speech_detected and silence_start is None:
                        silence_start = elapsed
                    if speech_detected and silence_start is not None:
                        if (elapsed - silence_start) >= silence_secs:
                            break
            break # stream ran to completion
        except sd.PortAudioError as e:
            print(f"\n [PortAudio error, attempt {attempt + 1}] {e}")
            if attempt == 0:
                time.sleep(0.5) # let CoreAudio release the device
                continue
            print(" Mic unavailable; saving silent recording and continuing.")

    print()

    if buffer:
        audio_native = np.concatenate(buffer).flatten().astype(np.float32) / 32768.0
        # resample to 16 kHz for Whisper
        audio_16k = librosa.resample(audio_native, orig_sr=native_rate, target_sr=LOCAL_SAMPLE_RATE)
        audio_int16 = (audio_16k * 32768.0).astype(np.int16)
    else:
        audio_int16 = np.zeros((0,), dtype=np.int16)

    # speech-detected flag so the transcription gate can use it
    global _local_speech_detected
    _local_speech_detected = speech_detected

    with wave.open(LOCAL_WAV, "wb") as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2)
        wf.setframerate(LOCAL_SAMPLE_RATE)
        wf.writeframes(audio_int16.tobytes())
    print(f" Recording saved ({elapsed:.1f}s, speech={'yes' if speech_detected else 'no'}).")

def local_say(text: str):
    "Speak text using host OS built-in TTS (macOS `say` or Windows SAPI)."
    text = clean_for_tts(text)
    try:
        if sys.platform == "win32":
            # Windows: SAPI via PowerShell; env-var avoids escaping
            subprocess.run(
                ["powershell", "-NoProfile", "-Command",
                 "Add-Type -AssemblyName System.Speech;(New-Object
                    ↪ System.Speech.Synthesis.SpeechSynthesizer).Speak($env:GAZE_TTS_TEXT)"],
                env={**os.environ, "GAZE_TTS_TEXT": text},
                check=True, timeout=30,
            )
        else:
            subprocess.run(["say", text], check=True, timeout=30)
```

```
    except Exception as e:
        print(f" Local TTS broke: {e}")

def say(ssh_tts, text):
    "Dispatch TTS to local or Pepper depending on mode."
    text = clean_for_tts(text)
    if LOCAL_MODE:
        local_say(text)
    else:
        nao_say(ssh_tts, text)
    time.sleep(0.5) # ensure text-to-speech (TTS) drains so it does not hear itself

def record(ssh, energy_threshold,
           no_speech_max: float = LOCAL_NO_SPEECH_MAX,
           silence_secs: float = LOCAL_SILENCE_SECS,
           record_max_secs: float = RECORD_MAX_SECS):
    """Dispatch recording to local or Pepper depending on mode;
       per-turn think-budget set by: AdaptiveEngine.recommend_think_budget().
       INPUT_IS_LOCAL routes the mic to the Mac even when LOCAL_MODE is False
       (gaze21 hybrid: Pepper-out, Mac-in)."""
    if INPUT_IS_LOCAL:
        local_record(max_secs=record_max_secs,
                     no_speech_max=no_speech_max,
                     silence_secs=silence_secs)
    else:
        nao_record(ssh, energy_threshold,
                   record_max_secs=record_max_secs,
                   silence_secs=silence_secs)

# gesture mapping; motions per game/emotional context
# tags: celebrate, encourage, think, wave, calm, energetic, neutral

GESTURE_CODE = {
    "wave": """
from naoqi import ALProxy
import time
m = ALProxy("ALMotion","127.0.0.1",9559)
m.setStiffnesses("RArm", 1.0)
safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
time.sleep(0.2)
m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
time.sleep(0.2)
m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
for _ in range(3):
    m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
    m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
time.sleep(0.4)
m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
m.setStiffnesses("RArm", 0.0)
""",

    "correct_wave": """
from naoqi import ALProxy
import time
m = ALProxy("ALMotion","127.0.0.1",9559)
m.setStiffnesses("RArm", 1.0)
safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
```

```
time.sleep(0.2)
m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
time.sleep(0.2)
m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
for _ in range(3):
    m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
    m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
time.sleep(0.4)
m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
m.setStiffnesses("RArm", 0.0)
""",
}


# LED COLOUR MAP
LED_COLOURS = {
    InferredState.THRIVING: 0x0000FF00, # green
    InferredState.COMFORTABLE: 0x0000FFFF, # cyan
    InferredState.STRUGGLING: 0x00FFFF00, # yellow
    InferredState.FRUSTRATED: 0x00FF0000, # red
    InferredState.DISENGAGED: 0x00FF00FF, # magenta
}


_STATE_COLOURS = {
    InferredState.THRIVING: "green",
    InferredState.COMFORTABLE: "blue",
    InferredState.STRUGGLING: "orange",
    InferredState.FRUSTRATED: "red",
    InferredState.DISENGAGED: "grey",
}

def nao_gesture(ssh, gesture_type: str):
    "Execute a gesture on Pepper aligned to the game context."
    code = GESTURE_CODE.get(gesture_type, GESTURE_CODE["wave"])
    try:
        nao_run(ssh, code)
    except Exception as e:
        print(f" Gesture {gesture_type!r} did not play: {e}")


def measure_volume() -> float:
    "RMS amplitude of the WAV; soundfile-based so multi-channel files collapse to mono cleanly."
    try:
        audio, _sr = sf.read(LOCAL_WAV, dtype="float32")
        if audio.size == 0:
            return 0.0
        if audio.ndim > 1:
            audio = audio.mean(axis=1)
        return float(np.sqrt(np.mean((audio * 32768.0) ** 2)))
    except Exception as e:
        print(f" Volume RMS calc failed: {e}")
        return 0.0

# Whisper hallucination blacklist; pre-normalised
# exact-match: short fillers that would false-positive if matched as substrings.
WHISPER_HALLUCINATION_EXACT = {
    "you", "mm", "hmm", "uh", "um",
    "thank you", "thanks",
```

```
    "      ",
    "       ",
    "     ",
    "            ",
    "        ",
}

# fragments: if any appears anywhere in the normalised text, treat as hallucination.
WHISPER_HALLUCINATION_FRAGMENTS = (
    "questions or comments",
    "questions or other problems",
    "post them in the comments",
    "in the comments section",
    "in the comment section",
    "if you like this video",
    "if you liked this video",
    "if you enjoyed the video",
    "if you enjoyed this video",
    "please subscribe",
    "subscribe and like",
    "like and subscribe",
    "like it and subscribe",
    "subscribe to our channel",
    "subscribe to my channel",
    "dont forget to subscribe",
    "do not forget to subscribe",
    "hit the like button",
    "smash that like button",
    "click the link",
    "link in the description",
    "link in the bio",
    "in the description below",
    "thanks for watching",
    "thank you for watching",
    "thank you so much for watching",
    "thanks so much for watching",
    "see you next time",
    "ill see you next time",
    "see you in the next video",
    "see you in the next one",
    # Whisper-prompt regurgitation; default prompt mentions "companion robot"
    "companion robot",
    "chats with a companion",
    "answers a quiz",
    "with a companion robot",
)

# back-compat alias
WHISPER_HALLUCINATIONS = WHISPER_HALLUCINATION_EXACT

def normalise_for_blacklist(text: str) -> str:
    "Normalise so blacklist matches independent of Whisper output."
    t = unicodedata.normalize("NFKC", text).lower()
    kept = [ch for ch in t if ch.isalnum() or ch.isspace()]
    return " ".join("".join(kept).split())

# regex blacklist; URL outros, promo boilerplate
_URL_HALLUCINATION = re.compile(r"\bhttps?://|www\.|\b\w+\.(?:com|org|net|au|co\.uk|google|sites)\b", re.I)
```

```python
_DISCLAIMER_HALLUCINATION = re.compile(r"\b(please see|visit|subscribe|like and subscribe|disclaimer|description)\b.{0,40}\b(complete|full|link|below|above)\b", re.I)

def is_known_hallucination(text: str) -> bool:
    norm = normalise_for_blacklist(text)
    if norm == "" or norm in WHISPER_HALLUCINATION_EXACT:
        return True
    # any fragment present anywhere in the normalised text counts
    if any(frag in norm for frag in WHISPER_HALLUCINATION_FRAGMENTS):
        return True
    # structural catch: Whisper leaks YouTube-description URLs on silence-amplified input
    if _URL_HALLUCINATION.search(text):
        return True
    if _DISCLAIMER_HALLUCINATION.search(text):
        return True
    return False

def has_real_speech(wav_path: str, min_speech_ms: int = 250,
                    threshold: float = 0.5) -> bool:
    "Silero VAD pre-gate; relaxed defaults so short answers (yes/Tom/six) survive."
    if _silero_model is None or get_speech_timestamps is None:
        return True
    try:
        wav = read_audio(wav_path, sampling_rate=LOCAL_SAMPLE_RATE)
        segments = get_speech_timestamps( # pass in the audio and model to return a list of dicts containing the start and end frame indices of genuine speech
            wav, _silero_model,
            sampling_rate=LOCAL_SAMPLE_RATE,
            min_speech_duration_ms=min_speech_ms,
            threshold=threshold,
            return_seconds=False,
        )
        return bool(segments)
    except Exception as e:
        print(f" Silero VAD check failed ({e}); falling through to Whisper")
        return True

def has_wake_word(wav_path: str) -> bool:
    """
    Vosk wake-word gate. True if "Pepper" or "Gaze" is heard in the WAV.
    Returns True if the Vosk model didn't load (optional dep).
    """
    if _vosk_model is None or KaldiRecognizer is None:
        return True
    try:
        rec = KaldiRecognizer(_vosk_model, LOCAL_SAMPLE_RATE,
                              json.dumps(["pepper", "gaze", "[unk]"]))
        with wave.open(wav_path, "rb") as wf:
            while True:
                data = wf.readframes(4000)
                if len(data) == 0:
                    break
                rec.AcceptWaveform(data)
        final = json.loads(rec.FinalResult())
        text = (final.get("text") or "").lower()
        return "pepper" in text or "gaze" in text
    except Exception as e:
        print(f" Vosk wake-word check failed ({e}); falling through to Whisper")
        return True
```

```python

#open ai whisperings and incase fails and in silent
def transcribe(bypass_wake_word: bool = False,
               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
               record_again=None,
               max_hallucination_retries=None) -> str:
    # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
    attempts_remaining = max_hallucination_retries
    while True:
        if INPUT_IS_LOCAL and not _local_speech_detected:
            return ""

        # Silero VAD hard gate; NAO + LOCAL
        if not has_real_speech(LOCAL_WAV):
            print(" Silero VAD found no speech; skipping Whisper.")
            return ""

        # Vosk wake-word gate; bypass for name prompt
        if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
            print(" No wake-word detected; skipping Whisper.")
            return ""

        try:
            with open(LOCAL_WAV, "rb") as fh:
                resp = client.audio.transcriptions.create(
                    model="whisper-1", #
                    file=fh, #
                    response_format="verbose_json", # get self-signals for hallucination detection
                    temperature=0.0, # zero randomness
                    prompt=whisper_prompt,
                    timeout=API_TIMEOUT,
                )
            text = (getattr(resp, "text", "") or "").strip()
            print(f" Whisper raw text: {text!r}")

            # Whisper self-signals from verbose_json
            segments = getattr(resp, "segments", None) or []
            suspected_hallucination = False
            if segments:
                no_speech_vals = [s.no_speech_prob for s in segments
                                  if getattr(s, "no_speech_prob", None) is not None]
                logprob_vals = [s.avg_logprob for s in segments
                                if getattr(s, "avg_logprob", None) is not None]
                compression_vals = [s.compression_ratio for s in segments
                                    if getattr(s, "compression_ratio", None) is not None]
                print(f" Whisper diagnostics: no_speech={no_speech_vals}, avg_logprob={logprob_vals}, compression={compression_vals}")
                if no_speech_vals and max(no_speech_vals) > 0.6:
                    print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f}); dropping {text!r}")
                    return ""
                if logprob_vals and min(logprob_vals) < -1.3:
                    print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f}); dropping {text!r}")
                    return ""
                # repetition-loop hallucination; flag for retry path
                if compression_vals and max(compression_vals) > 2.4:
                    print(f" Whisper repetition loop (max compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
```

```
                    hallucination")
                suspected_hallucination = True

            # normalised hallucination blacklist + Whisper-self-signal hallucinations
            if suspected_hallucination or is_known_hallucination(text):
                print(f" Filtered Whisper hallucination: {text!r}")
                if record_again is not None and (attempts_remaining is None or attempts_remaining > 0):
                    if attempts_remaining is not None: # whilst more retries remain decrement each time-taken and re-record
                        attempts_remaining -= 1
                        print(f" Disregarding hallucination; listening again (attempts left: {attempts_remaining}).")
                    else:
                        print(f" Disregarding hallucination; listening again.")
                    record_again()
                    continue
                return ""

            # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks "Pepperoni"/"Gazebo" false positives..
            stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[,.\s]*', '', text).strip()
            return stripped
        except Exception as e:
            print(f" Whisper transcribe failed ({e}); returning empty")
            return ""

# facial-expression pipeline

def capture_thread_loop(camera):
    """Tight capture loop; just grab the latest frame.
    Decoupled from detection so the dashboard runs at native fps."""
    global _preview_frame
    while True:
        ret, frame = camera.read()
        if not ret: # if camera read fails keep old frame as preview
            time.sleep(0.03)
            continue
        with _preview_lock:
            _preview_frame = frame

def detect_thread_loop(face_model, face_cascade):
    """Run cascade + CNN on a downscaled copy of the latest frame; update cached bbox + emotion.
    6-7 Hz is enough for a turn-based consumer."""
    global _last_emotion, _last_bbox
    while True:
        time.sleep(DETECT_INTERVAL_SECS)
        with _preview_lock:
            frame = _preview_frame
        if frame is None:
            continue

        # 2x downscale; 4x faster cascade
        small = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
        gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        if len(faces) == 0:
            bbox = None
```

```
            emotion, conf = "Neutral", 0.0
        else:
            x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
            roi = gray[y:y+h, x:x+w]
            resized = cv2.resize(roi, (48, 48))
            inp = resized[np.newaxis, :, :, np.newaxis]
            emotion, conf = face_model.predict(inp)
            # scale 'bbox' back to raw-frame coordinates (was shrunk 2×)
            bbox = (x*2, y*2, w*2, h*2)

        with _preview_lock:
            _preview_state["emotion"] = emotion
            _preview_state["confidence"] = conf
            _last_bbox = bbox
        _last_emotion = emotion

def start_preview_thread(camera, face_model, face_cascade):
    "Start both the capture and the detection daemon threads."
    cap_t = threading.Thread(target=capture_thread_loop,
                             args=(camera,),
                             daemon=True)
    det_t = threading.Thread(target=detect_thread_loop,
                             args=(face_model, face_cascade),
                             daemon=True)
    cap_t.start()
    det_t.start()
    return cap_t, det_t

def capture_and_classify(ssh, face_model, face_cascade,
                         local_camera=None) -> tuple[str, float]:
    "Return (emotion, confidence). Local-camera branch reads + classifies inline; NAO branch reads cached state set by detect_thread_loop off the live Pepper video stream."
    # local-camera + preview thread already running: just read the cache
    if DEBUG_PREVIEW and local_camera is not None:
        with _preview_lock:
            return _preview_state["emotion"], _preview_state["confidence"]

    # NAO: read cached state from detect thread
    if local_camera is None:
        with _preview_lock:
            return _preview_state["emotion"], _preview_state["confidence"]

    # local-camera per-turn classification (DEBUG_PREVIEW disabled)
    ret, frame = local_camera.read()
    if not ret:
        print(" [Local camera failed] cv2.VideoCapture.read() returned False")
        return "Neutral", 0.0

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    if len(faces) == 0:
        bbox = None
        emotion, conf = "Neutral", 0.0
    else:
        x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
        roi = gray[y:y+h, x:x+w]
        resized = cv2.resize(roi, (48, 48))
        inp = resized[np.newaxis, :, :, np.newaxis]
```

```python
        emotion, conf = face_model.predict(inp)
        bbox = (x, y, w, h)

    global _preview_frame, _last_bbox
    with _preview_lock:
        _preview_state["emotion"] = emotion
        _preview_state["confidence"] = conf
        _last_bbox = bbox
        _preview_frame = frame

    return emotion, conf


API_TIMEOUT = 10 # 10-second timeout; prevents Pepper-freeze if OpenAI/network stalls

def check_answer(user_answer: str, correct_answer: str,
                 question_context: str) -> bool:
    "Verify the user's answer via gpt."
    if not user_answer.strip():
        return False

    try:
        resp = client.chat.completions.create(
            model="gpt-4.1",
            messages=[{
                "role": "system",
                "content": "You are an answer checker. Given a question, the correct answer, and the user's spoken answer, determine if the user is correct. Be lenient with pronunciation, phrasing, and partial answers that demonstrate knowledge. Respond with ONLY 'correct' or 'incorrect'.",
            }, {
                "role": "user",
                "content": f"""Question: {question_context}
                    Correct answer: {correct_answer}
                    User's answer: {user_answer}""",
            }],
            temperature=0.0,
            timeout=API_TIMEOUT,
        )
        verdict = resp.choices[0].message.content.strip().lower()
        return verdict == "correct" # if AI says its 'correct' return verdict=correct
    except Exception as e:
        print(f" Answer verifier API failed: {e}")
        fallback_answer = correct_answer.lower().strip()
        fallback_user = user_answer.lower().strip()

        return fallback_user == fallback_answer

# save/load sessions

def save_session(engine: AdaptiveEngine, preferred_game: Optional[GameType] = None,
                 quiet: bool = False):
    """Save session progress so user can continue later.
       quiet=True suppresses the "Session saved to ..." print, used when saving
       after every round so the log doesn't fill up with save confirmations.
    """
    data = {
        "total_correct": engine.total_correct,
        "total_rounds_played": engine.total_rounds_played,
```

```
        "best_streak": engine.best_streak,
        "games_played": {g.value: c for g, c in engine.games_played.items()},
        "game_switches": engine.game_switch_count,
        "last_difficulty": engine.current_difficulty.value,
        "last_game": engine.current_game.value,
        "preferred_game": preferred_game.value if preferred_game else None,
        "rounds_played": len(engine.history),
        "recent_questions": engine.recent_questions[-30:],
        "recent_answers": engine.recent_answers[-30:],
        "recent_game_types": engine.recent_game_types[-30:],
        "round_log": [
            {
                "round": r.round_number,
                "game": r.game_type.value,
                "difficulty": r.difficulty.value,
                "correct": r.correct,
                "time": round(r.response_time, 1),
                "expression": r.facial_expression,
                "vocal": r.vocal_emotion,
                "state": r.inferred_state.value,
            }
            for r in engine.history
        ],
    }
    with open(SAVE_FILE, "w") as f:
        json.dump(data, f, indent=2)
    if not quiet:
        print(f" Session saved to {SAVE_FILE}")

def load_session() -> Optional[dict]:
    "Load a previously saved session, if one exists."
    if not os.path.exists(SAVE_FILE):
        return None
    try:
        with open(SAVE_FILE, "r") as f:
            return json.load(f)
    except (json.JSONDecodeError, IOError) as e:
        print(f" Save file corrupt, ignoring: {e}")
        return None

def restore_engine(save_data: dict) -> AdaptiveEngine:
    "Restore engine state from saved data."
    engine = AdaptiveEngine()
    engine.total_correct = save_data.get("total_correct", 0)
    # legacy fallback; older saves used rounds_played instead
    engine.total_rounds_played = save_data.get("total_rounds_played",
                                         save_data.get("rounds_played", 0))
    engine.best_streak = save_data.get("best_streak", 0)
    engine.game_switch_count = save_data.get("game_switches", 0)
    engine.current_difficulty = Difficulty(save_data.get("last_difficulty", 2))
    engine.current_game = GameType(save_data.get("last_game", "numbers"))
    engine.recent_questions = list(save_data.get("recent_questions", []))[-30:]
    engine.recent_answers = list(save_data.get("recent_answers", []))[-30:]
    engine.recent_game_types = list(save_data.get("recent_game_types", []))[-30:]
    for g_val, count in save_data.get("games_played", {}).items():
        try:
            engine.games_played[GameType(g_val)] = count
        except ValueError as e:
            print(f" Could not restore game type {g_val!r}: {e}")
```

```python
    return engine

def delete_save():
    "Remove the save file after a completed session or on user request."
    if os.path.exists(SAVE_FILE):
        os.remove(SAVE_FILE)

# OpenAI function-calling + conversation helpers
'''
    - generate_game_question: called by the LLM when it wants to ask a new question; returns a question + correct answer for the specified game type and difficulty
    - check_game_answer: called by the LLM after the user answers a question; returns answer correctness if so

    - evaluate_last_adaptation: called by the LLM to self-evaluate if the previous round's adaptive strategy did indeed help the user

    - request_more_time: called by the LLM when the user asks for more time
'''
TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "generate_game_question",
            "description": "Generate a new countdown-style game question. Call this when the conversation naturally leads to playing a game.",
            "parameters": {
                "type": "object",
                "properties": {
                    "game_type": {
                        "type": "string",
                        "enum": ["numbers", "letters"],
                        "description": "Type of countdown game round.",
                    },
                    "difficulty": {
                        "type": "string",
                        "enum": ["EASY", "MEDIUM", "HARD"],
                        "description": "Difficulty level for the question.",
                    },
                },
                "required": ["game_type", "difficulty"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "check_game_answer",
            "description": "Verify whether the user's spoken answer to the current game question is correct. Call this after the user gives an answer to an active game question.",
            "parameters": {
                "type": "object",
                "properties": {
                    "user_answer": {
                        "type": "string",
                        "description": "What the user said.",
                    },
                    "correct_answer": {
                        "type": "string",
```

```
                        "description": "The known correct answer.",
                    },
                    "question_context": {
                        "type": "string",
                        "description": "The original question text for context.",
                    },
                },
                "required": ["user_answer", "correct_answer", "question_context"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "evaluate_last_adaptation",
            "description": "Self-evaluate whether the previous round's adaptive strategy actually helped the user. Returns a natural-language evaluation.",
            "parameters": {"type": "object", "properties": {}},
        },
    },
    {
        "type": "function",
        "function": {
            "name": "request_more_time",
            "description": "The user has asked for more time to think about the current game question. Acknowledge their request warmly.",
            "parameters": {"type": "object", "properties": {}},
        },
    },
]

def build_signal_context(engine: AdaptiveEngine,
                         expression: str, expr_conf: float,
                         vocal_emo: str, vocal_conf: float,
                         vol_rms: float, response_time: float) -> str:
    "Build a signal summary string injected alongside every user message so the LLM can adapt its behaviour to the user's real-time emotional state without explicit adaptive-engine instructions."
    correctness = engine.rolling_correctness()
    recent_faces = [r.facial_expression for r in engine.history[-3:]]
    recent_vocal = [r.vocal_emotion for r in engine.history[-3:]]

    # think-budget (float) -> (str) word label for LLM
    if engine.think_budget_secs >= 17.0:
        pacing = "relaxed and patient"
    elif engine.think_budget_secs <= 13.0:
        pacing = "brisk and energetic"
    else:
        pacing = "standard"

    lines = [
        "--- LIVE SIGNALS ---",
        f"Turn: {engine.round_number}",
        f"Face: {expression} ({expr_conf:.0%}) [PRIMARY signal: trust this first]",
        f"Voice: {vocal_emo} ({vocal_conf:.0%}) [advisory only; the vocal model is noisy and often stuck on 'fearful']",
        f"Volume: {vol_rms:.0f} RMS",
        f"Response time: {response_time:.1f}s",
        f"Rolling accuracy (last {CORRECTNESS_WINDOW}): {correctness:.0%}", # overall correctness
```

```python
        f"System pacing: {pacing}",
    ]
    if recent_faces:
        lines.append(f"Recent faces: {', '.join(recent_faces)}")
    if recent_vocal:
        lines.append(f"Recent vocal: {', '.join(recent_vocal)}")

    return "\n".join(lines)

def converse(conversation: list, tools: list) -> object:
    "General-conversation OpenAI `gpt-5.4` chat-completion call with function calling."
    try:
        resp = client.chat.completions.create(
            model="gpt-5.4",
            messages=conversation,
            tools=tools,
            temperature=0.8, # bit of randomness/creativity for more interesting companion
            timeout=API_TIMEOUT,
        )
        return resp.choices[0].message
    except Exception as e:
        print(f"converse() API failed... : {e}")
        return types.SimpleNamespace(
            content="I had a brief network hiccup. Let's keep going! [gesture:think]",
            tool_calls=None, role="assistant")

def execute_tool_call(tool_name: str, tool_args: dict,
                      engine: AdaptiveEngine, game_state: GameState,
                      conversation: list,
                      preferred_game: Optional[GameType],
                      dashboard=None) -> str:
    "Dispatch a function-calling tool invocation and return a JSON string result."
    if tool_name == "generate_game_question":
        gt = tool_args.get("game_type", "numbers")
        diff = tool_args.get("difficulty", "MEDIUM")
        result = generate_game_question_internal(
            gt, diff,
            recent=engine.recent_questions,
            recent_answers=engine.recent_answers,
            recent_game_types=engine.recent_game_types,
        )
        # dedup: question + answer + game_type
        new_q = result.get("question", "").strip()
        new_a = str(result.get("answer", "")).strip()
        if new_q:
            engine.recent_questions.append(new_q)
            engine.recent_questions = engine.recent_questions[-30:]
        if new_a:
            engine.recent_answers.append(new_a)
            engine.recent_answers = engine.recent_answers[-30:]
        engine.recent_game_types.append(gt)
        engine.recent_game_types = engine.recent_game_types[-30:]
        # sync adaptive engine with LLM's chosen difficulty so the engine's next decide() starts from correct baseline
        try:
            engine.current_difficulty = Difficulty[diff]
        except (KeyError, ValueError):
            pass
        # also sync current_game; record_round logs game_type from this
```

```
        try:
            engine.current_game = GameType(gt)
        except ValueError:
            pass
        game_state.active = True
        game_state.current_question = result.get("question", "")
        game_state.current_answer = result.get("answer", "")
        game_state.category = result.get("category", "")
        return json.dumps(result)

    elif tool_name == "check_game_answer":
        ua = tool_args.get("user_answer", "")
        if not game_state.active:
            game_state.last_answer_checked = False
            return json.dumps({"correct": False, "error": "no active game question"})
        # snapshot before same-chain regen overwrites
        game_state.answered_question = game_state.current_question
        game_state.answered_answer = game_state.current_answer
        game_state.answered_game_type = engine.current_game
        game_state.answered_difficulty = engine.current_difficulty
        # Python state is the source of truth, not LLM-supplied args
        is_correct = check_answer(
            ua,
            game_state.current_answer,
            game_state.current_question,
        )
        # last_answer_checked: ensures record_round() runs
        game_state.last_answer_checked = True
        game_state.last_answer_correct = is_correct
        if is_correct:
            game_state.active = False
        return json.dumps({"correct": is_correct})

    elif tool_name == "evaluate_last_adaptation":
        evaluation = engine.evaluate_adaptation()
        return json.dumps({"evaluation": evaluation})

    elif tool_name == "request_more_time":
        game_state.waiting = True
        return json.dumps({"acknowledged": True})

    else:
        return json.dumps({"error": f"Unknown tool: {tool_name}"})

def process_llm_response(message, conversation: list,
                         engine: AdaptiveEngine, game_state: GameState,
                         preferred_game: Optional[GameType],
                         dashboard=None) -> str:
    "Handle the LLM response, including any tool call chains."
    msg_dict = {"role": "assistant", "content": message.content or ""}
    if message.tool_calls: # if tool calls are required inject the function-call into convo history so LLM can decide persistence
        msg_dict["tool_calls"] = [
            {
                "id": tc.id,
                "type": "function",
                "function": {"name": tc.function.name, "arguments": tc.function.arguments},
            }
            for tc in message.tool_calls # iterate over each tool call in the message
```

```
        ]
    conversation.append(msg_dict)
    # Cap recursive tool calls at 5 rounds to prevent infinite loops
    max_tool_rounds = 5
    current_msg = message
    used_answer_check = False # gate same-chain regen so engine.decide can adapt difficulty first
    for _ in range(max_tool_rounds):
        if not current_msg.tool_calls:
            break # kill loop if no more tool calls

        # block same-batch regen; engine.decide() must update difficulty first
        batch_has_check = any(tc.function.name == "check_game_answer"
                              for tc in current_msg.tool_calls)
        for tc in current_msg.tool_calls:
            fn_name = tc.function.name
            try:
                fn_args = json.loads(tc.function.arguments)
            except json.JSONDecodeError:
                fn_args = {}

            if batch_has_check and fn_name == "generate_game_question":
                result_str = json.dumps({"skipped": True,
                                         "reason": "deferred until after adaptive decision"})
                print(f" Skipping {fn_name}: deferred after answer check")
            else:
                print(f" Calling tool {fn_name}({fn_args})")
                result_str = execute_tool_call(
                    fn_name, fn_args, engine, game_state,
                    conversation, preferred_game, dashboard
                )
                print(f" Tool returned: {result_str[:120]}")
            if fn_name == "check_game_answer":
                used_answer_check = True
            conversation.append({
                "role": "tool",
                "tool_call_id": tc.id,
                "content": result_str,
            })
        # withhold generate_game_question after a check; defer to next turn so engine.decide can update difficulty first
        next_tools = TOOLS
        current_msg = converse(conversation, next_tools)
        resp_dict = {"role": "assistant", "content": current_msg.content or ""}
        if current_msg.tool_calls:
            resp_dict["tool_calls"] = [
                {
                    "id": tc.id,
                    "type": "function",
                    "function": {"name": tc.function.name, "arguments": tc.function.arguments},
                }
                for tc in current_msg.tool_calls
            ]
        conversation.append(resp_dict)

    return current_msg.content or ""

#look for gestures
def extract_gesture(text: str) -> str:
    "Returns wave always -only gesture now used."
```

```
    return "wave"
def validate_numbers_round(numbers: list, target: int) -> bool:
    """Skipped -too slow for real-time use. gpt is instructed to verify its own answer."""
    return True


def generate_game_question_internal(game_type_str: str, difficulty_str: str,
                               recent: Optional[list[str]] = None,
                               recent_answers: Optional[list[str]] = None,
                               recent_game_types: Optional[list[str]] = None) -> dict:
    try:
        gt = GameType(game_type_str)
    except ValueError:
        gt = GameType.NUMBERS
    try:
        diff = Difficulty[difficulty_str]
    except (KeyError, ValueError):
        diff = Difficulty.MEDIUM

    import secrets
    variety_seed = secrets.token_hex(3)

    banned_questions = "\n".join(f"- {q}" for q in (recent or [])[-30:])
    banned_answers = ", ".join((recent_answers or [])[-30:])

    prompt = f"""You are generating a {game_type_str.upper()} round for a Countdown-style game.

        GAME TYPE: {game_type_str.upper()} -do not generate any other type.
        DIFFICULTY: {DIFFICULTY_DESCRIPTIONS[diff]}
        VARIETY SEED (use this to pick different numbers/letters every time): {variety_seed}

        STRICT RULES -you MUST follow all of these:
        1. This MUST be a {game_type_str.upper()} round. No exceptions.
        2. The question MUST be completely different from every question in the BANNED LIST below.
        3. The correct answer MUST NOT appear in the BANNED ANSWERS list below.
        4. Pick a fresh number set or letter set -do not reuse combinations you have used before.

        BANNED LIST (do not repeat or closely mimic any of these):
        {banned_questions if banned_questions else "none yet"}

        BANNED ANSWERS (your answer must not be any of these):
        {banned_answers if banned_answers else "none yet"}

        Respond with a JSON object only -no markdown, no code fences:
        {{"question": "...", "answer": "...", "category": "..."}}
    """

    try:
        resp = client.chat.completions.create(
            model="gpt-5.4",
            messages=[
                        {"role": "system", "content": (
                    "You generate countdown-style game questions. Respond ONLY with valid JSON. "
                    "Follow all rules in the user prompt exactly. "
                    "For numbers rounds you MUST verify your arithmetic before responding. "
                    "The 'answer' field must contain ONLY the final solution expression -"
                    "for example '50 + 25 = 75'. "
                    "Do NOT include working out, alternative attempts, explanations, "
                    "or commentary in the answer field. One clean expression only."
```

```
                )},
                {"role": "user", "content": prompt},

            ],
            temperature=1.2,
            timeout=API_TIMEOUT,
        )
        content = resp.choices[0].message.content.strip()
        if content.startswith("```"):
            content = content.split("\n", 1)[1] if "\n" in content else content[3:]
            if content.endswith("```"):
                content = content[:-3]
            content = content.strip()
        result = json.loads(content)

        # validate numbers rounds as gpt normally hallucinates unreachable targets
        if game_type_str == "numbers":
            try:
                import re
                q = result.get("question", "")
                nums = list(map(int, re.findall(r'\b\d+\b', q.split("make")[0])))
                target_match = re.search(r'make.*?(\d+)', q)
                if target_match and nums:
                    target = int(target_match.group(1))
                    if not validate_numbers_round(nums, target):
                        print(f"gpt produced unreachable target {target} from {nums}; using fallback")
                        return {
                            "question": "Using 25, 50, 3, 6, 4, and 2, make the target 100.",
                            "answer": "25 x 4 = 100",
                            "category": "numbers fallback",
                        }
            except Exception as ve:
                print(f" Numbers validation skipped: {ve}")

        return result
    except json.JSONDecodeError:
        return {"question": content if 'content' in locals() else "", "answer": "", "category":
            ↪ "general"}
    except Exception as e:
        print(f" Game question gen failed: {e}")
        return {
            "question": "Let's try a quick one: what is 25 + 17?",
            "answer": "42",
            "category": "arithmetic fallback",
        }

_STATE_COLOURS = {
    InferredState.THRIVING: "green",
    InferredState.COMFORTABLE: "blue",
    InferredState.STRUGGLING: "orange",
    InferredState.FRUSTRATED: "red",
    InferredState.DISENGAGED: "grey",
}

class GazeDashboard:
    """Tkinter dashboard for GAZE.
    1- create a window with two panels: left = conversation then video, right for inferred
        ↪ stats/signals/state
    2- left panel: camera canvas above the scrollable conversation log; log is read-only default
```

```
    ↪ and
    flipped to "normal" when a new line is appended
3- right panel: round/score/streak counters at top, then the transcription block (what the
    ↪ user said + correct/incorrect,
    recoloured per outcome), then the live signals (face CNN, voice MLP, volume RMS, response
        ↪ time, rolling accuracy,
    think budget), then the adaptive decision block (inferred state, difficulty, tone,
        ↪ adaptations, plus a grey eval-note line below)
4- quit handling: the Quit button, Esc, Cmd/Ctrl-Q and the window-close (X) all converge on
    ↪ quit_app(), so the SSH farewell + LED-off
    always run no matter how the user closes the dashboard
5- refresh loops: camera_refresh() composites the latest raw frame with the cached detection
    ↪ overlay at native fps; signal_refresh()
    repaints the StringVar-bound labels at a slower cadence; both self-reschedule via
        ↪ root.after()
"""

CAMERA_W, CAMERA_H = 400, 300

def __init__(self, ssh=None, ssh_tts=None):
    # stashed for quit_app farewell + LED-off
    self._ssh = ssh
    self._ssh_tts = ssh_tts
    self.root = tk.Tk()
    self.root.title("GAZE Dashboard")
    self.root.resizable(False, False)

    # left col: video + chat; right col: stats
    left = tk.Frame(self.root)
    left.grid(row=0, column=0, padx=8, pady=8, sticky="n")

    # camera canvas; black letterboxes empty pixels
    self.camera_label = tk.Label(left, bg="black",
                                 width=self.CAMERA_W, height=self.CAMERA_H)
    self.camera_label.pack()

    tk.Label(left, text="Conversation:").pack(anchor="w", pady=(6, 0))
    conv_frame = tk.Frame(left)
    conv_frame.pack()
    # read-only by default; helpers flip to insert
    self._conv_text = tk.Text(conv_frame, font=("Courier", 10),
                              width=50, height=10, wrap="word",
                              state="disabled")
    # scrollbar bidirectional wiring
    conv_scroll = tk.Scrollbar(conv_frame, command=self._conv_text.yview)
    self._conv_text.configure(yscrollcommand=conv_scroll.set)
    self._conv_text.pack(side="left", fill="both", expand=True)
    conv_scroll.pack(side="right", fill="y")

    right = tk.Frame(self.root)
    right.grid(row=0, column=1, padx=(0, 8), pady=8, sticky="n")

    tk.Label(right, text="GAZE (Game-Adaptive Zone of Engagement) Dashboard",
             font=("TkDefaultFont", 14, "bold")).pack(pady=(0, 4))

    self._round_var = tk.StringVar(value="Round: -")
    self._score_var = tk.StringVar(value="Score: 0/0")
    self._streak_var = tk.StringVar(value="Streak: 0")
    for var in (self._round_var, self._score_var, self._streak_var):
```

```
            tk.Label(right, textvariable=var).pack(anchor="w")

        # user transcription only; answer on stdout
        tk.Label(right, text="").pack()
        tk.Label(right, text="Transcription:",
                 font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
        self._heard_var = tk.StringVar(value="You said: -")
        self._result_var = tk.StringVar(value="")
        tk.Label(right, textvariable=self._heard_var).pack(anchor="w")
        # fg recolour on correctness
        self._result_label = tk.Label(right, textvariable=self._result_var,
                                      font=("TkDefaultFont", 11, "bold"))
        self._result_label.pack(anchor="w")

        tk.Label(right, text="").pack()
        tk.Label(right, text="Live Signals:",
                 font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
        self._face_var = tk.StringVar(value="Face (CNN): -")
        self._voice_var = tk.StringVar(value="Voice (MLP): -")
        self._vol_var = tk.StringVar(value="Volume RMS: -")
        self._time_var = tk.StringVar(value="Response time: -")
        self._acc_var = tk.StringVar(value="Rolling acc: -")
        self._budget_var = tk.StringVar(value="Think budget: -")
        for var in (self._face_var, self._voice_var, self._vol_var,
                    self._time_var, self._acc_var, self._budget_var):
            tk.Label(right, textvariable=var,
                     font=("Courier", 10)).pack(anchor="w")

        tk.Label(right, text="").pack()
        tk.Label(right, text="Adaptive Decision:",
                 font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
        self._state_var = tk.StringVar(value="State: -")
        # kept for state-coloured fg
        self._state_label = tk.Label(right, textvariable=self._state_var,
                                     font=("TkDefaultFont", 11, "bold"))
        self._state_label.pack(anchor="w")

        self._diff_var = tk.StringVar(value="Difficulty: -")
        self._tone_var = tk.StringVar(value="Tone: -")
        self._adapt_var = tk.StringVar(value="Adaptations: -")
        for var in (self._diff_var, self._tone_var, self._adapt_var):
            tk.Label(right, textvariable=var).pack(anchor="w")

        # adaptation-eval note (miniscule 'grey' text)
        self._eval_var = tk.StringVar(value="")
        self._eval_label = tk.Label(right, textvariable=self._eval_var,
                                    font=("TkDefaultFont", 9), fg="grey",
                                    wraplength=340, justify="left")
        self._eval_label.pack(anchor="w")

        tk.Button(right, text="Quit (Esc)",
                  command=self.quit_app).pack(pady=(10, 0))

        self.root.bind("<Escape>", lambda e: self.quit_app())
        # Cmd-Q is macOS-only; guard each
        for combo in ("<Command-q>", "<Control-q>"):
            try:
                self.root.bind(combo, lambda e: self.quit_app())
            except tk.TclError:
```

```
                pass
        # route window-close through quit_app
        self.root.protocol("WM_DELETE_WINDOW", self.quit_app)

        self.camera_refresh()
        self.signal_refresh()
        self.root.update()                   # paint before mainloop()


    def camera_refresh(self):
        """Composite the latest raw frame with the cached detection overlay.
        Bbox is drawn on every fresh raw frame for a live-feed."""
        with _preview_lock:
            frame = _preview_frame
            bbox = _last_bbox
            emotion = _preview_state["emotion"]
            conf = _preview_state["confidence"]

        if frame is not None:
            display = frame.copy()                          # don't mutate the shared raw frame; the bbox/text overlays are per-paint
            if bbox is not None:
                x, y, w, h = bbox
                cv2.rectangle(display, (x, y), (x + w, y + h), (0, 255, 0), 2)
            label = f"{emotion} ({conf:.0%})"
            cv2.putText(display, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                        0.8, (0, 255, 0), 2)
            rgb = cv2.cvtColor(display, cv2.COLOR_BGR2RGB) # OpenCV is BGR, Tk/PIL expect RGB; swap channels here
            small = cv2.resize(rgb, (self.CAMERA_W, self.CAMERA_H))
            img = ImageTk.PhotoImage(Image.fromarray(small)) # Tk-compatible image wrapper around the PIL image
            self.camera_label.configure(image=img)
            self.camera_label._photo = img # prevent garbage collection -Tk doesn't keep a ref to PhotoImage so without an attribute hold the image gets GC'd and the canvas blanks

        self.root.after(33, self.camera_refresh) # ~30 fps target -Tk's standard self-rescheduling animation pattern; after(ms, fn) queues fn onto the mainloop after ms milliseconds

    def signal_refresh(self):
        with _preview_lock:
            emotion = _preview_state["emotion"]
            conf = _preview_state["confidence"]
        self._face_var.set(f"Face (CNN): {emotion} ({conf:.0%})")
        self.root.after(500, self.signal_refresh) # 2 Hz

    # thread-safe scheduling: tkinter widgets are main-thread only (macOS); root.after(0, fn) queues fn onto the main loop.

    def on_main(self, fn):
        "Schedule fn to run on the main (tkinter) thread."
        try:
            self.root.after(0, fn)
        except tk.TclError:
            pass


    def update_robot_speech(self, text: str):
        # flip read-only widget to insert, scroll, re-disable
```

```
        def apply():
            self._conv_text.configure(state="normal")
            self._conv_text.insert("end", f"Robot: {text}\n\n")
            self._conv_text.see("end")
            self._conv_text.configure(state="disabled")
        self.on_main(apply) # schedule the UI update on the main thread

    def append_user_speech(self, text: str):
        def apply():
            self._conv_text.configure(state="normal")
            self._conv_text.insert("end", f"You: {text}\n")
            self._conv_text.see("end")
            self._conv_text.configure(state="disabled")
            self._heard_var.set(f"You said: {text}") # mirror to right panel
        self.on_main(apply)

    def update_think_budget(self, secs: float):
        "Update the dashboard's adaptive think-budget diagnostic row."
        def apply():
            self._budget_var.set(f"Think budget: {secs:.1f}s")
        self.on_main(apply)

    def update_signals(self, round_num: int, user_answer: str, correct_answer: str,
                       correct: bool, expression: str, expr_conf: float,
                       vocal_emo: str, vocal_conf: float, vol_rms: float,
                       response_time: float, rolling_acc: float,
                       total_correct: int, total_rounds: int, streak: int):
        def apply():
            self._round_var.set(f"Round: {round_num}")
            self._score_var.set(f"Score: {total_correct}/{total_rounds}")
            self._streak_var.set(f"Streak: {streak}")

            self._heard_var.set(f"You said: {user_answer if user_answer else '(silence)'}") # extract user's answer
            if not correct_answer: # baseline; non-active game state
                self._result_var.set("")
                self._result_label.configure(fg="grey")
            elif correct:
                self._result_var.set("CORRECT")
                self._result_label.configure(fg="green")
            elif not user_answer:
                self._result_var.set("NO ANSWER")
                self._result_label.configure(fg="grey")
            else:
                self._result_var.set("INCORRECT")
                self._result_label.configure(fg="red")

            self._face_var.set(f"Face (CNN): {expression} ({expr_conf:.0%})")
            self._voice_var.set(f"Voice (MLP): {vocal_emo} ({vocal_conf:.0%})")

            vol_tag = "(loud)" if vol_rms > 2000 else "(quiet)" if vol_rms < VOLUME_THRESHOLD else "(normal)"
            bar_len = min(int(vol_rms / 250), 20)
            meter = "#" * bar_len + "." * (20 - bar_len)
            self._vol_var.set(f"Volume RMS: {vol_rms:>5.0f} [{meter}] {vol_tag}")
            self._time_var.set(f"Response time: {response_time:.1f}s")
            self._acc_var.set(f"Rolling acc: {rolling_acc:.0%}")

        self.on_main(apply)
```

```
    def update_decision(self, decision, adaptation_eval: str = None):
        state = decision.inferred_state
        def apply():
            self._state_var.set(f"State: {state.value.upper()}")
            self._state_label.configure(fg=_STATE_COLOURS.get(state, "grey"))
            self._diff_var.set(f"Difficulty: {decision.difficulty.name}")
            self._tone_var.set(f"Tone: {decision.tone}")
            flags = []
            if decision.give_hint: flags.append("hint")
            if decision.give_encouragement: flags.append("encouragement")
            if decision.switch_game: flags.append(f"switch -> {decision.game_type.value}")
            self._adapt_var.set(f"Adaptations: {', '.join(flags) if flags else 'none'}")
            self._eval_var.set(adaptation_eval if adaptation_eval else "")
        self.on_main(apply)

    def quit_app(self):
        "Speak a farewell, dim Pepper's eye-LEDs, then kill the process."
        print("\n [Dashboard] Quit requested.")
        if not LOCAL_MODE and self._ssh is not None:
            nao_set_leds(self._ssh, "FaceLeds", 0x00000000, 0.5)
        # local_say in LOCAL_MODE; SSH-TTS otherwise
        say(self._ssh_tts, "It was great to chat! See you next time!")
        self.close()
        os._exit(0)

    def close(self):
        try:
            self.root.destroy()
        except tk.TclError:
            pass

def is_goodbye(text: str) -> bool:
    "LLM goodbye-intent gate; True if the user signalled they want to leave."
    if not text:
        return False
    try:
        bye = client.chat.completions.create(
            model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
            timeout=API_TIMEOUT,
            messages=[{"role": "system", "content":
                "Did the user just signal they want to end the conversation (e.g. 'bye', 'goodbye', 'I'm done', 'see you later', 'I have to go')? Reply ONLY with 'GOODBYE' or 'CONTINUE'."},
                {"role": "user", "content": text}],
        ).choices[0].message.content.strip().upper()
    except Exception:
        return False
    return "GOODBYE" in bye

def conversation_loop(dashboard, face_model, face_cascade, speech_model,
                      local_camera, ssh, ssh_tts, energy_threshold):
    "Comprehensive main-conversation loop."
    preferred_game = None
    engine = AdaptiveEngine()
    game_state = GameState()


    save_data = load_session()
```

```
    if save_data:
        # legacy fallback; older saves only had per-session rounds_played
        prev_rounds = save_data.get("total_rounds_played",
                                    save_data.get("rounds_played", 0))
        prev_correct = save_data.get("total_correct", 0)

        welcome_back = f"Welcome back! Last time you played {prev_rounds} rounds and got {prev_correct} correct. Want to continue where you left off, or start fresh?"
        if not LOCAL_MODE:
            nao_track_face(ssh, enable=True)
            nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
            nao_gesture(ssh, "wave")
        say(ssh_tts, welcome_back)
        print(f"\nRobot: {welcome_back}")
        dashboard.update_robot_speech(welcome_back)

        print("\nListening for continue/fresh...")
        if not LOCAL_MODE:
            nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
        record(ssh, energy_threshold,
               no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0)

        expression, expr_conf = capture_and_classify(
            ssh, face_model, face_cascade, local_camera)
        vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
        vol_rms = measure_volume()
        dashboard.update_signals(
            round_num=0, user_answer="", correct_answer="", correct=False,
            expression=expression, expr_conf=expr_conf,
            vocal_emo=vocal_emo, vocal_conf=vocal_conf,
            vol_rms=vol_rms, response_time=0, rolling_acc=0,
            total_correct=0, total_rounds=0, streak=0,
        )

        resume_text = transcribe(
            bypass_wake_word=True,
            record_again=lambda: record(ssh, energy_threshold,
                                        no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0),
        )
        if resume_text:
            print(f" Heard: {resume_text}")
            dashboard.append_user_speech(resume_text)
            if is_goodbye(resume_text):
                dashboard.quit_app()
        else:
            print(" Heard: (silence)")

        # LLM intent classifier; on failure fall back to a keyword check, then
        # default to CONTINUE so an API timeout never silently nukes the save.
        intent = ""
        if resume_text:
            try:
                intent = client.chat.completions.create(
                    model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
                    timeout=API_TIMEOUT,
                    messages=[{"role": "system", "content":
                        "The user was just asked whether they want to CONTINUE their previous session or start FRESH. Reply ONLY with 'CONTINUE' or 'FRESH'."},
                        {"role": "user", "content": resume_text}],
```

```
                ).choices[0].message.content.strip().upper()
            except Exception as e:
                print(f" Resume intent classification failed ({e}); falling back to keyword match")
                low = resume_text.lower()
                fresh_kw = ("fresh", "start over", "restart", "new game", "start again", "begin again", "from scratch")
                continue_kw = ("continue", "carry on", "keep going", "resume", "where i left", "where we left", "yes", "yeah", "yep")
                if any(k in low for k in fresh_kw):
                    intent = "FRESH"
                elif any(k in low for k in continue_kw):
                    intent = "CONTINUE"
                else:
                    # ambiguous; preserve the save rather than wipe it
                    intent = "CONTINUE"
                print(f" Keyword fallback resolved intent: {intent}")
        if "CONTINUE" in intent:
            engine = restore_engine(save_data)
            preferred_game = engine.current_game
            restored_rounds = save_data.get("total_rounds_played",
                                            save_data.get("rounds_played", 0))
            restore_msg = f"Restoring your previous session: {restored_rounds} rounds on record."
            say(ssh_tts, restore_msg)
            print(f"\nRobot: {restore_msg}")
        else:
            delete_save()
            fresh_msg = "No worries, starting fresh! Your previous save has been cleared."
            say(ssh_tts, fresh_msg)
            print(f"\nRobot: {fresh_msg}")


    if not LOCAL_MODE:
        nao_track_face(ssh, enable=True)
        nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)

    conversation = [{"role": "system", "content": BASE_SYSTEM_PROMPT}]

    greeting_directive = "[Give a warm, supportive greeting. Mention you can chat, play games, or just hang out. Keep it to 2 sentences.]"
    greeting_msg = converse(
        conversation + [{"role": "user", "content": greeting_directive}],
        TOOLS,
    )
    greeting_text = greeting_msg.content or "Hello, lovely to meet you!"
    greeting_gesture = extract_gesture(greeting_text)
    greeting_speech = re.sub(r'\[gesture:\w+\]', '', greeting_text).strip()

    conversation.append({"role": "assistant", "content": greeting_text})

    if not LOCAL_MODE:
        nao_gesture(ssh, "wave")
    say(ssh_tts, greeting_speech)
    print(f"\nRobot: {greeting_speech}")
    dashboard.update_robot_speech(greeting_speech)


    turn_count = 0
    # tiered disengagement; check-in at 3, game-switch at 4
    nudge_level = 0
```

```
    try:
        while True:
            turn_count += 1
            print(f"\n{'-' * 40} Turn {turn_count} {'-' * 40}")

            print("Capturing expression...")
            expression, expr_conf = capture_and_classify(
                ssh, face_model, face_cascade, local_camera
            )
            if expr_conf < FACE_CONFIDENCE_THRESHOLD:
                print(f" Expression: {expression} ({expr_conf:.2f}): LOW CONFIDENCE, treating as Neutral")
                expression = "Neutral"
            else:
                print(f" Expression: {expression} ({expr_conf:.2f})")

            question_start = time.time()
            print("Listening...")
            if not LOCAL_MODE:
                # brief cyan pulse on the face LEDs so the user can see the robot is actively listening; ears also go green
                nao_set_leds(ssh, "FaceLeds", 0x0000FFFF, 0.2)
                nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)

            # adaptive think-budget based on previous round's state + current face
            prev_state = (engine.history[-1].inferred_state if engine.history
                          else InferredState.COMFORTABLE)
            prev_rt = engine.history[-1].response_time if engine.history else 0.0
            no_speech_max, silence_secs, record_max_secs = engine.recommend_think_budget(
                state=prev_state, expression=expression,
                prev_response_time=prev_rt,
                consecutive_silences=engine.consecutive_silences,
                waiting=game_state.waiting,
            )
            dashboard.update_think_budget(record_max_secs)

            record(ssh, energy_threshold,
                   no_speech_max=no_speech_max,
                   silence_secs=silence_secs,
                   record_max_secs=record_max_secs)
            response_time = time.time() - question_start

            # extended think-budget consumed; reset for next turn
            if game_state.waiting:
                game_state.waiting = False

            vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
            if vocal_conf < VOICE_CONFIDENCE_THRESHOLD:
                print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f}): LOW CONFIDENCE, treating as neutral")
                vocal_emo = "neutral"
            else:
                print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f})")

            vol_rms = measure_volume()
            print(f" Volume RMS: {vol_rms:.0f}")

            # (c) transcribe; per-chunk gate, file-wide RMS removed
```

```python
            relisten = lambda: record(ssh, energy_threshold,
                                  no_speech_max=no_speech_max,
                                  silence_secs=silence_secs,
                                  record_max_secs=record_max_secs)
            if INPUT_IS_LOCAL: # hybrid check
                if _local_speech_detected:
                    user_text = transcribe(bypass_wake_word=True, record_again=relisten)
                else:
                    print(f" No speech chunk detected (floor={LOCAL_SILENCE_RMS}); skipping transcription.")
                    user_text = ""
            elif vol_rms >= NAO_MIN_RMS_TO_TRANSCRIBE:
                print(f" NAO RMS diagnostic: {vol_rms:.0f} (gate floor {NAO_MIN_RMS_TO_TRANSCRIBE})")
                user_text = transcribe(bypass_wake_word=True, record_again=relisten)
            else:
                print(f" WAV near-empty ({vol_rms:.0f} < {NAO_MIN_RMS_TO_TRANSCRIBE}); skipping transcription.")
                user_text = ""

            if user_text:
                print(f" Heard: {user_text}")
                dashboard.append_user_speech(user_text)
                # user spoke; reset silence + nudge so future silence re-arms from scratch
                engine.consecutive_silences = 0
                nudge_level = 0

                if is_goodbye(user_text):
                    dashboard.quit_app()
            else:
                print(" Heard: (silence)")
                dashboard.append_user_speech("(silence)")
                engine.consecutive_silences += 1

                # surface signals to dashboard even though the LLM is skipped this turn
                dashboard.update_signals(
                    round_num=turn_count, user_answer="", correct_answer="",
                    correct=False, expression=expression, expr_conf=expr_conf,
                    vocal_emo=vocal_emo, vocal_conf=vocal_conf,
                    vol_rms=vol_rms, response_time=response_time,
                    rolling_acc=engine.rolling_correctness(),
                    total_correct=engine.total_correct,
                    total_rounds=engine.total_rounds_played,
                    streak=engine.consecutive_correct,
                )

                # 1- check-in at 3 silences (proposal: "intervenes to re-engage them")
                if engine.consecutive_silences >= 3 and nudge_level < 1:
                    nudge_prompt = [
                        {"role": "system",
                         "content": BASE_SYSTEM_PROMPT},
                        {"role": "user",
                         "content": "[The user has been quiet for 3 turns. Offer ONE brief, gentle check-in; no question, no nagging. 1 sentence max.]"},
                    ]
                    nudge_msg = converse(nudge_prompt, [])
                    nudge_text = (nudge_msg.content or "").strip()
                    nudge_speech = re.sub(r'\[gesture:\w+\]', '', nudge_text).strip()
                    if not LOCAL_MODE:
                        # recolour eyes to signal the state has shifted; user sees the shift even if
```

```
                        they say nothing back
                        nao_set_leds(ssh, "FaceLeds",
                                     LED_COLOURS[InferredState.DISENGAGED], 0.4)
                    if nudge_speech:
                        say(ssh_tts, nudge_speech)
                        print(f"\nRobot (nudge): {nudge_speech}")
                        dashboard.update_robot_speech(nudge_speech)
                    nudge_level = 1

                # 2- switch game, soft re-invitation
                elif engine.consecutive_silences >= 4 and nudge_level < 2:
                    engine.current_game = engine.pick_different_game()
                    engine.game_switch_count += 1
                    escalate_prompt = [
                        {"role": "system",
                         "content": BASE_SYSTEM_PROMPT},
                        {"role": "user",
                         "content": f"[The user has gone silent for 4 turns. Warmly acknowledge they've gone quiet then offer a {engine.current_game.value} round as a soft alternative. Two sentences total.]"},
                    ]
                    esc_msg = converse(escalate_prompt, [])
                    esc_text = (esc_msg.content or "").strip()
                    esc_speech = re.sub(r'\[gesture:\w+\]', '', esc_text).strip()
                    if not LOCAL_MODE:
                        nao_set_leds(ssh, "FaceLeds",
                                     LED_COLOURS[InferredState.DISENGAGED], 0.4)
                    if esc_speech:
                        say(ssh_tts, esc_speech)
                        print(f"\nRobot (escalate): {esc_speech}")
                        dashboard.update_robot_speech(esc_speech)
                    nudge_level = 2

                continue # skip the LLM call; just wait for the user

            if user_text.lower().strip() in [
                "stop", "quit", "exit", "goodbye", "bye", "end",
                "i want to stop", "let's stop", "no more",
            ]:
                print("Patient wants to stop.")
                break

            signal_ctx = build_signal_context(
                engine, expression, expr_conf,
                vocal_emo, vocal_conf, vol_rms, response_time,
            )

            user_msg_content = f"{signal_ctx}\n\nUser says: {user_text}"
            conversation.append({"role": "user", "content": user_msg_content})

            # trim conversation to prevent context overflow; keep system + last 40 messages (20 exchanges)
            if len(conversation) > 42:
                conversation = [conversation[0]] + conversation[-40:]

            if not LOCAL_MODE:
                nao_set_leds(ssh, "EarLeds", 0x000000FF, 0.3) # blue = thinking

            llm_message = converse(conversation, TOOLS)
```

```
            response_text = process_llm_response(
                llm_message, conversation, engine, game_state,
                preferred_game, dashboard
            )

            if not response_text.strip():
                response_text = "I'm here! What would you like to talk about? [gesture:neutral]"

            gesture_type = extract_gesture(response_text)
            speech_text = re.sub(r'\[gesture:\w+\]', '', response_text).strip()

            if not LOCAL_MODE:
                if engine.adaptation_log:
                    last_state_str = engine.adaptation_log[-1]["state"]
                    try:
                        last_state = InferredState(last_state_str)
                    except ValueError:
                        last_state = InferredState.COMFORTABLE
                    nao_set_leds(
                        ssh, "FaceLeds",
                        LED_COLOURS.get(last_state, 0x00FFFFFF), 0.5
                    )

            say(ssh_tts, speech_text)

            print(f"\nRobot: {speech_text}")
            if game_state.current_answer:
                print(f"(Game answer: {game_state.current_answer})")
            dashboard.update_robot_speech(speech_text)

            # (k) update dashboard (signals + decision if game active); use the actual answer check result from the tool chain
            was_game_answer = game_state.last_answer_checked
            correct = game_state.last_answer_correct if was_game_answer else False

            if not LOCAL_MODE and was_game_answer and correct:
                threading.Thread(target=nao_gesture, args=(ssh, "correct_wave"), daemon=True).start()

            # only run engine.decide on real game answers; chat or more-time turns mustn't ramp difficulty or pollute streak counters
            if was_game_answer:
                # use snapshot so a same-chain generate_game_question can't pollute this round's logged values
                answered_q = game_state.answered_question or game_state.current_question
                answered_a = game_state.answered_answer or game_state.current_answer
                answered_gt = game_state.answered_game_type or engine.current_game
                answered_df = game_state.answered_difficulty or engine.current_difficulty
                decision = engine.decide(
                    expression, expr_conf, response_time,
                    correct=correct,
                    answer_text=user_text,
                    vocal_emotion=vocal_emo, vocal_conf=vocal_conf,
                    volume_rms=vol_rms,
                )
                engine.record_round(RoundResult(
                    round_number=turn_count,
                    game_type=answered_gt,
                    difficulty=answered_df,
```

```
                    question=answered_q,
                    user_answer=user_text,
                    correct=correct,
                    response_time=response_time,
                    facial_expression=expression,
                    expression_confidence=expr_conf,
                    vocal_emotion=vocal_emo,
                    vocal_emotion_confidence=vocal_conf,
                    volume_rms=vol_rms,
                    inferred_state=decision.inferred_state,
                ))
                # save after every round; power-cycle loses one turn, not five
                save_session(engine, preferred_game, quiet=True)
                dashboard.update_signals(
                    round_num=turn_count, user_answer=user_text,
                    correct_answer=answered_a,
                    correct=correct,
                    expression=expression, expr_conf=expr_conf,
                    vocal_emo=vocal_emo, vocal_conf=vocal_conf,
                    vol_rms=vol_rms, response_time=response_time,
                    rolling_acc=engine.rolling_correctness(),
                    total_correct=engine.total_correct,
                    total_rounds=engine.total_rounds_played,
                    streak=engine.consecutive_correct,
                )
                dashboard.update_decision(decision)
                # clear snapshot now the round is logged
                game_state.answered_question = ""
                game_state.answered_answer = ""
                game_state.answered_game_type = None
                game_state.answered_difficulty = None
            else:
                # chat or more-time turn; refresh dashboard signals only
                dashboard.update_signals(
                    round_num=turn_count, user_answer=user_text,
                    correct_answer="",
                    correct=False,
                    expression=expression, expr_conf=expr_conf,
                    vocal_emo=vocal_emo, vocal_conf=vocal_conf,
                    vol_rms=vol_rms, response_time=response_time,
                    rolling_acc=engine.rolling_correctness(),
                    total_correct=engine.total_correct,
                    total_rounds=engine.total_rounds_played,
                    streak=engine.consecutive_correct,
                )

            game_state.last_answer_checked = False

            # auto-save every 2 turns
            if turn_count % 2 == 0:
                save_session(engine, preferred_game, quiet=True)
                print(" [Auto-save]")

    except KeyboardInterrupt:
        print("\n\nInterrupted.")


    save_session(engine, preferred_game)

```

```
    summary = engine.get_session_summary()
    print(f"\n{'=' * 60}")
    print(" SESSION SUMMARY")
    print(f"{'=' * 60}")
    for k, v in summary.items():
        print(f" {k}: {v}")

    if summary.get("rounds", 0) > 0:
        acc = summary["accuracy"]
        streak_note = (f" Your best streak was {summary['best_streak']} in a row!"
                       if summary["best_streak"] >= 3 else "")
        if acc >= 0.8:
            farewell = f"Amazing session! You got {summary['correct']} out of {summary['rounds']} right.{streak_note} Brilliant work! Your progress is saved; see you next time!"
        elif acc >= 0.5:
            farewell = f"Great effort! You scored {summary['correct']} out of {summary['rounds']}.{streak_note} Well played! Your progress is saved; see you next time!"
        else:
            farewell = f"Thanks for playing! You got {summary['correct']} out of {summary['rounds']}.{streak_note} Every round is a learning opportunity. Your progress is saved; see you next time!"
    else:
        farewell = "It was great to chat! Your progress is saved; see you next time!"

    if not LOCAL_MODE:
        nao_gesture(ssh, "wave")
    say(ssh_tts, farewell)
    print(f"\nRobot: {farewell}")

    if local_camera is not None:
        local_camera.release()
    if not LOCAL_MODE:
        nao_track_face(ssh, enable=False)
        nao_set_leds(ssh, "FaceLeds", 0x00000000, 0.5)
        ssh.close()
        ssh_tts.close()
    dashboard.close()
    print("\nGAZE disconnected.")

    # --------------------------------- MAIN ENTRY-POINT ---------------------------------

def main():
    print("----------------------------------------")
    print(" GAZE: Game-Adaptive Zone of Engagement")
    print(" Adaptive Game-System Ran on Pepper")
    print("----------------------------------------")

    print("\nLoading facial expression model...")
    face_model = FacialExpressionModel(MODEL_JSON, MODEL_WEIGHTS)
    face_cascade = cv2.CascadeClassifier(HAAR_CASCADE)
    print(" Facial model loaded.")

    speech_model = None
    if os.path.exists(SPEECH_MODEL):
        print("Loading speech emotion model...")
        speech_model = SpeechEmotionModel(SPEECH_MODEL)
        print(" Speech model loaded.")
    else:
```

```
        print(f" Speech emotion model not found at {SPEECH_MODEL}; vocal signal disabled.")
        print(" Run train_speech_model.py to generate it.")

    local_camera = None
    if USE_LOCAL_CAMERA:
        local_camera = cv2.VideoCapture(0)
        # BUFFERSIZE=1; else OpenCV buffers stale frames
        local_camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        local_camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        local_camera.set(cv2.CAP_PROP_FPS, 30)
        local_camera.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        print(" Using local webcam for expression detection.")
        if DEBUG_PREVIEW:
            start_preview_thread(local_camera, face_model, face_cascade)
            print(" Preview thread started.")
    else:
        print(" Using Pepper's camera for expression detection.")

    # connect to Pepper (skipped in local mode)
    ssh = None
    ssh_tts = None
    energy_threshold = DEFAULT_ENERGY_THRESHOLD

    # Pepper SSH connection: required whenever output goes through the robot
    # (TTS, LEDs, gestures, face-tracking). LOCAL_MODE remains the gate for that.
    if LOCAL_MODE:
        print("\n LOCAL MODE: skipping Pepper connection.")
    else:
        print(f"\nConnecting to Pepper at {NAO_IP}...")
        ssh = ssh_connect()
        ssh_tts = ssh_connect()     # dedicated TTS connection
        print(" Connected.")

        # Pepper stream ~10 fps; detect at 6-7 Hz
        if not USE_LOCAL_CAMERA:
            print(" Starting Pepper camera stream...")
            threading.Thread(target=pepper_video_loop,
                             args=(ssh,), daemon=True).start()
            threading.Thread(target=pepper_camera_receive_loop,
                             args=(NAO_IP,), daemon=True).start()
            threading.Thread(target=detect_thread_loop,
                             args=(face_model, face_cascade), daemon=True).start()
            print(" Pepper camera stream threads started.")

    # Mic-side ambient calibration: pick whichever microphone INPUT_IS_LOCAL says.
    # In gaze21 hybrid mode this is the Mac mic even though Pepper is connected.
    if INPUT_IS_LOCAL:
        print("\nCalibrating Mac microphone ambient noise level...")
        local_calibrate_ambient()
    else:
        print("\nCalibrating ambient noise level (stay quiet for 3 seconds)...")
        energy_threshold = nao_calibrate_ambient(ssh)

    dashboard = GazeDashboard(ssh=ssh, ssh_tts=ssh_tts)
    print(" Dashboard launched.")

    conv_thread = threading.Thread(
        target=conversation_loop,
        args=(dashboard, face_model, face_cascade, speech_model,
```

```
            local_camera, ssh, ssh_tts, energy_threshold),
        daemon=True,
    )
    conv_thread.start()

    # tkinter mainloop on main thread (macOS-required); keep camera preview and GUI responsive whilst the conversation loop blocks on I/O
    dashboard.root.mainloop()

if __name__ == "__main__":
    main()
```